

Triggering Based Real Time Radar Signal Processing on Edge

Final Hardware Implementation Report

Aditya Gautam Aditya Kak Kabeer Kundra
Nadish Gulati

Indraprastha Institute of Information Technology Delhi

May 5, 2026

Abstract

This project implements a complete radar signal-processing pipeline on the Zynq Processing System (PS) of a Xilinx Zynq SoC. Two different waveform schemes are studied and compared: a Golay-sequence-based single-carrier radar and a Zadoff-Chu (ZC) sequence-based multi-carrier radar. The pipeline covers signal generation, channel simulation (delay, Doppler, AWGN), matched filtering (correlation), and two-dimensional Range-Doppler map computation.

1. System overview

1.1 Radar principles

A monostatic radar collocates the transmitter and receiver. A known waveform is transmitted; the echo reflected by a target arrives delayed by a time proportional to the round-trip distance, and frequency-shifted by an amount proportional to the target's radial velocity (the Doppler effect). If the delay is τ and the radial velocity is v , then:

$$R = \frac{c \cdot \tau}{2} \tag{1}$$

$$f_d = \frac{2 \cdot v \cdot f_c}{c} \tag{2}$$

where c is the speed of light and f_c is the carrier frequency. These two quantities are estimated jointly from a 2-D Range-Doppler map computed by the signal processor.

1.2 System parameters

The following are the system parameters specified:

Table 1: System and Simulation Parameters

Parameter	Value
Centre Frequency (f_c)	30 GHz
Bandwidth (B)	300 MHz
Range Resolution ($\Delta R = c/2B$)	0.5 m
Maximum Range	256 m
Velocity Resolution	2.86 m/s
Maximum Unambiguous Velocity	± 732.42 m/s
Number of Samples per Pulse ($NSAMP$)	512
FIFO Size	32,768 samples
Number of Batches	16
Total Pulses — Golay (TP_G)	256 (pairs)
Total Pulses — ZC (TP_Z)	512
Monte Carlo Trials	50 per SNR point
SNR Range Tested	−45 dB to −30 dB

1.3 System Architecture

The current implementation runs entirely on the Zynq Processing System (PS) as a software-only simulation, with hardware deployment planned as a subsequent phase. The architecture comprises of three logical subsystems:

- **Transmitter:** The PS generates the radar waveform, either a Golay complementary pair or a Zadoff–Chu (ZC) sequence, producing a baseband pulse ready for transmission.
- **Channel (Software Loopback):** Rather than passing through physical RF hardware, the transmitted waveform is looped back internally within the PS. A software channel model applies a configurable delay (corresponding to a target range), a Doppler frequency shift (corresponding to a target radial velocity), and additive white Gaussian noise (AWGN) at a controlled SNR, thereby emulating a realistic propagation environment in simulation.
- **Receiver:** The PS processes the noisy, delayed, and Doppler-shifted loopback signal by performing pulse compression via matched filtering (correlation with the known waveform), followed by a 1-D FFT across successive pulses to extract Doppler information, and finally constructs the 2-D Range-Doppler map for target detection and parameter estimation.

This software-loopback approach allows full validation of the waveform design, signal processing chain, and detection performance across a range of SNR conditions prior to hardware integration. In the planned hardware implementation, the software channel model will be replaced by a Programmable Logic (PL) implementation, with DMA controllers managing data transfer between the PS and the PL fabric.

2. Waveform generation

2.1 Golay Sequence: Single Carrier Radar

Golay complementary pairs are two binary sequences (G_a, G_b) of equal length whose aperiodic autocorrelation functions (ACFs) sum to a perfect impulse. Intuitively, the ACF measures how similar a sequence is to a shifted copy of itself; sidelobes in the ACF correspond to false target responses in radar. The key property of a Golay pair is that whatever sidelobes G_a produces, G_b produces equal and opposite ones, so their sum cancels completely:

$$R_{G_a}[k] + R_{G_b}[k] = 2N\delta[k] \quad (3)$$

where N is the sequence length, k is the lag index, and $\delta[k] = 1$ only at $k = 0$ and is zero everywhere else. This makes Golay pairs well-suited for pulse-compression radar, where sidelobe-free range profiles are desirable.

Sequence Generation Starting from the seed pair:

$$G_a = [+1, +1], \quad G_b = [+1, -1]$$

the sequences are grown recursively by concatenation:

$$G_a^{(k+1)} = \begin{bmatrix} G_a^{(k)} \\ G_b^{(k)} \end{bmatrix}, \quad G_b^{(k+1)} = \begin{bmatrix} G_a^{(k)} \\ -G_b^{(k)} \end{bmatrix} \quad (4)$$

Each iteration doubles the length. After $\log_2(512) - 1 = 8$ iterations, both sequences reach a length of $N = 1024$ samples.

Matched Filtering Once the received signal is captured, each pulse is correlated against the known transmitted template to determine where in time (i.e. at what range) the echo arrived. This correlation is computed efficiently in the frequency domain via the FFT. The outputs from the G_a and G_b pulses are then summed, invoking the sidelobe-cancellation property. This process is repeated across all $TP_G = 256$ transmitted pulse pairs, producing a 1024×256 compressed range-profile matrix.

Doppler Processing

A target moving radially imparts a small but consistent phase shift onto each successive pulse. A TP_G -point FFT applied along the pulse dimension at each range bin converts these accumulated phase shifts into Doppler frequency estimates. After an `fftshift` to centre zero velocity, the result is the 1024×256 Range-Doppler map RDM_G .

2.2 Zadoff–Chu Sequence: Multi Carrier Radar

Zadoff–Chu (ZC) sequences are complex sequences where every sample has the same magnitude (unimodular), with the useful property of zero periodic autocorrelation sidelobes and a flat spectrum. This means the signal energy is spread evenly across both time and frequency,

making ZC sequences well suited for OFDM (Orthogonal Frequency Division Multiplexing) based radar where the signal occupies multiple subcarriers simultaneously.

Sequence Generation The ZC sequence of length N is defined by the closed-form expression:

$$x[n] = \exp\left(-j\pi u \frac{n^2}{N}\right), \quad 0 \leq n \leq N-1 \quad (5)$$

where $N = 512$ is the number of subcarriers and $u = 1$ is the root index. Unlike the Golay case, no recursive construction is needed — the entire sequence is generated directly from this formula.

To handle the effect of the channel, a cyclic prefix (CP) is prepended to the sequence before transmission. The CP is a copy of the last N_{CP} samples of the sequence appended to the front, making the total transmitted frame length $N + N_{CP} = 1024$ samples.

Its purpose is to convert the linear convolution introduced by the channel into a circular convolution, which can be efficiently inverted using FFT-based processing.

Matched Filtering At the receiver, the cyclic prefix is first stripped, leaving a 512-sample block. Because the CP ensures the channel acts as a circular convolution, matched filtering can be performed entirely in the frequency domain:

1. A 512-point FFT is applied:

$$R = \mathcal{F}\{r[n]\}$$

2. The received spectrum is multiplied element-wise by the complex conjugate of the transmitted spectrum:

$$\hat{C}[k] = R[k] \cdot \mathcal{F}\{x[n]\}^*[k] \quad (6)$$

3. A 512-point IFFT recovers the compressed range profile:

$$c[n] = \mathcal{F}^{-1}\{\hat{C}[k]\} \quad (7)$$

This is repeated for each of the $TP_Z = 512$ received pulses, filling a 512×512 compressed range-profile matrix.

Doppler Processing As with the Golay case, a target's radial velocity causes a slow phase progression across successive pulses. A TP_Z -point FFT applied along the pulse dimension at each range bin extracts Doppler information. After an `fftshift` to centre zero velocity, the result is the final 512×512 Range-Doppler map RDM_{ZC} .

3. Twiddle Factor Calculation

To generate the precomputed twiddle factors, we used a simple Python script with the `numpy` library. Since our FFT size is $N = 512$, the lookup table requires half of that amount, which is 256 values.

The script runs a loop to calculate the phase angle for each index k using the standard formula

$-2\pi k/N$. It computes the real part using the cosine of the angle and the imaginary part using the sine of the angle. These values are then cast to 32-bit floating point numbers to match our hardware precision requirements.

Finally, the script automatically formats and prints the results as a C-style constant array named `TWIDDLE_ROM`. This allows us to easily copy and paste the generated table directly into our C code.

```
import numpy as np

N = 512
NUM = N // 2 # 256

print("const complex_tw_t TWIDDLE_ROM[256] = {")

for k in range(NUM):
    angle = -2 * np.pi * k / N
    real = np.cos(angle)
    imag = np.sin(angle)

    real = np.float32(real)
    imag = np.float32(imag)

    print(f"    {{{real:.10f}}f, {imag:.10f}}f},")

print("};")
```

HLS IP Development

4. 512-point FFT IP

4.1 Code Architecture

The top level function of the design is `fft_512_top`. This function uses AXI4-Stream interfaces for both input and output data. The use of AXI streams allows continuous and standard data transfer. First, the function reads an array of 512 complex float numbers from the input stream and stores them into a local memory buffer. It then reads and discards the next 512 samples from the stream.

After storing the data, the top function calls the core processing block named `fft_fast`. The `fft_fast` function performs an in-place Radix-2 Fast Fourier Transform. The architecture of this core function is divided into two main stages:

- **Bit Reversal:** The input array is reordered using a predefined lookup table. If the reversed index is greater than the current index, the elements in the array are swapped.

- **Butterfly Stages:** The code loops through the stages of the FFT. It computes the complex multiplication of the input data with the twiddle factors. Then, it performs the addition and subtraction required for the standard butterfly calculation.

Once the core function finishes processing, the top function writes the 512 updated samples back to the output stream. It also carefully sets the `last` signal to 1 for the final sample to indicate the end of the packet.

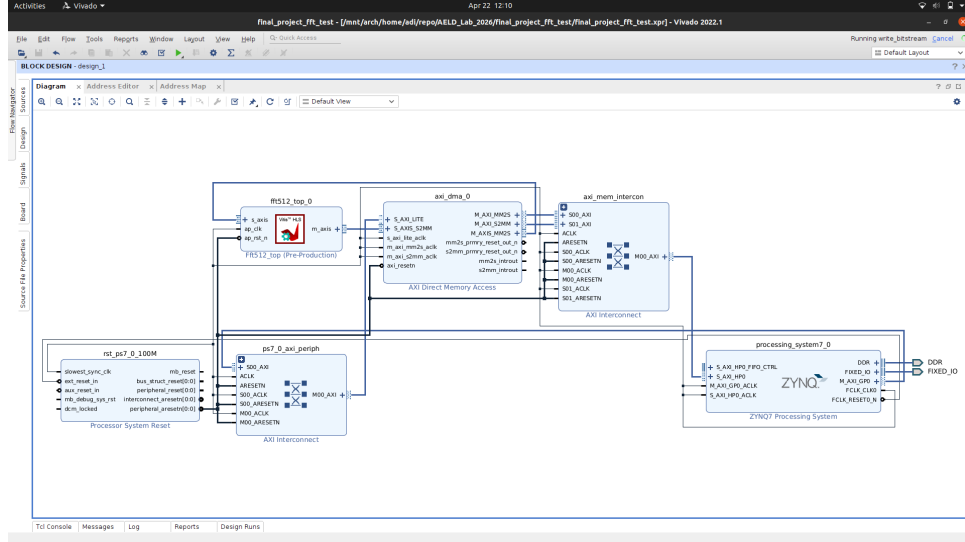


Figure 1: Testing Block Diagram

4.2 Optimizations and Results

In the first iteration of the code, the FFT was implemented without dedicated precomputed tables for the twiddle factors. This initial implementation achieved a speedup of 16.01.

To improve the design, we applied a major optimization. We shifted the twiddle factors into precomputed lookup tables. This optimization stops the hardware from performing complex trigonometric calculations on the fly. The optimized version with precomputed tables achieved a speedup of 5.1.

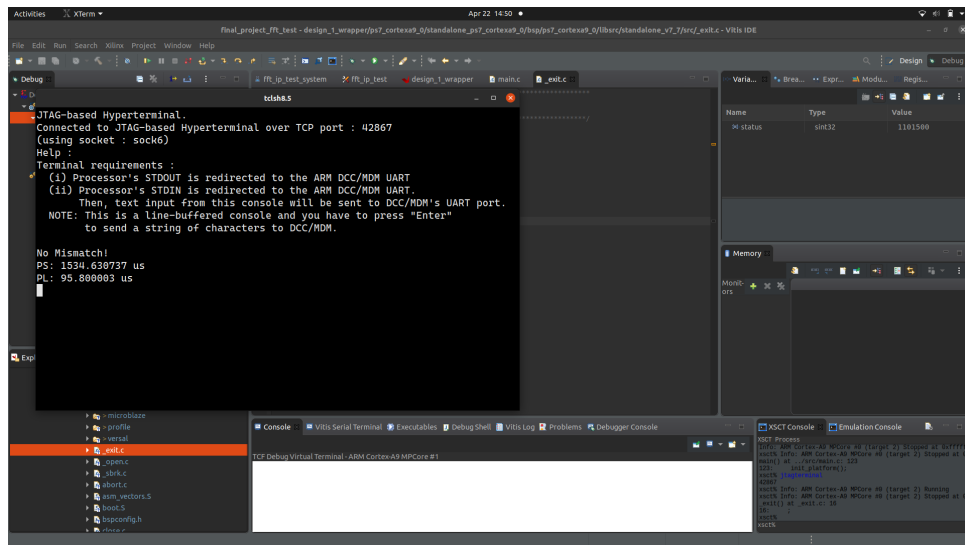


Figure 2: FFT Performance Without Precomputation (16.01 Speedup)

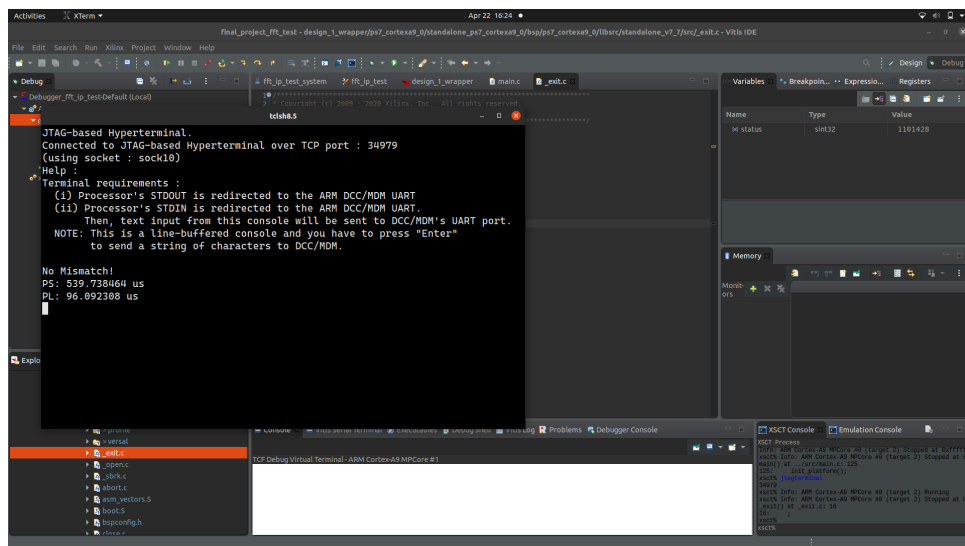


Figure 3: FFT Performance With Precomputation (5.1 Speedup)

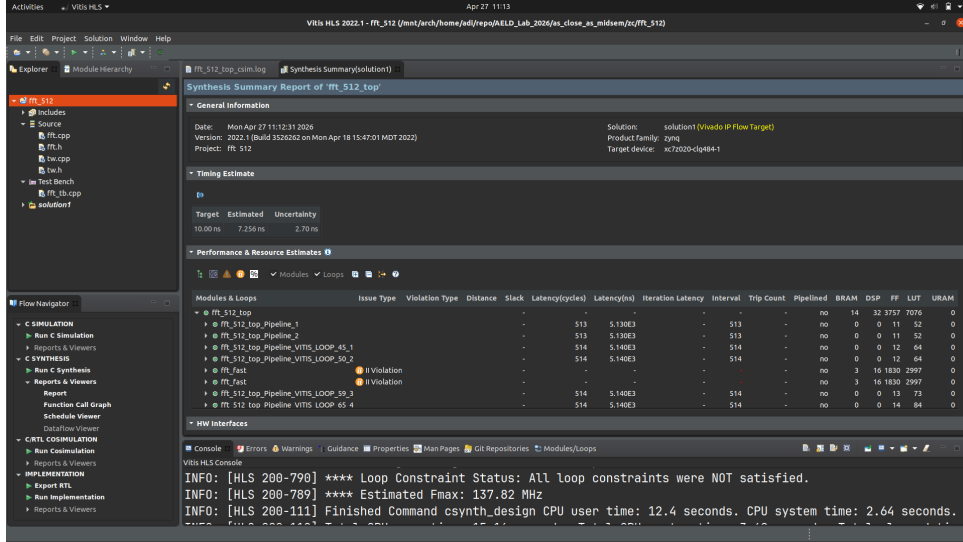


Figure 4: Latency results of the FFT IP

5. 512-point IFFT IP

5.1 Inverse FFT Architecture

The top level function for the Inverse Fast Fourier Transform is `ifft_512_top`. Just like the forward FFT, it uses AXI4 Stream interfaces to handle data transfer. The function reads exactly 512 complex samples from the input stream into a local array. Unlike the forward FFT function, it does not read and discard an extra set of samples.

The data is then passed to the core function named `ifft_fast`. The architecture of this function is very similar to the forward FFT. It executes the bit reversal and the butterfly computing stages using the same structure. The key difference in the inverse FFT is the final normalization step. After all the butterfly stages are finished, a loop multiplies every element in the array by the inverse of the length, which is $1/512$.

After the `ifft_fast` function completes, the top level function writes the 512 scaled samples back to the output stream and sets the `last` signal on the final sample.

5.2 IFFT Optimizations and Results

For the inverse FFT, we moved straight to the optimized version. We directly implemented the code using the precomputed twiddle tables and did not build a baseline version without them.

During testing, this optimized IFFT implementation achieved a speedup of 5.4. The figure below shows the performance results for this testing block.

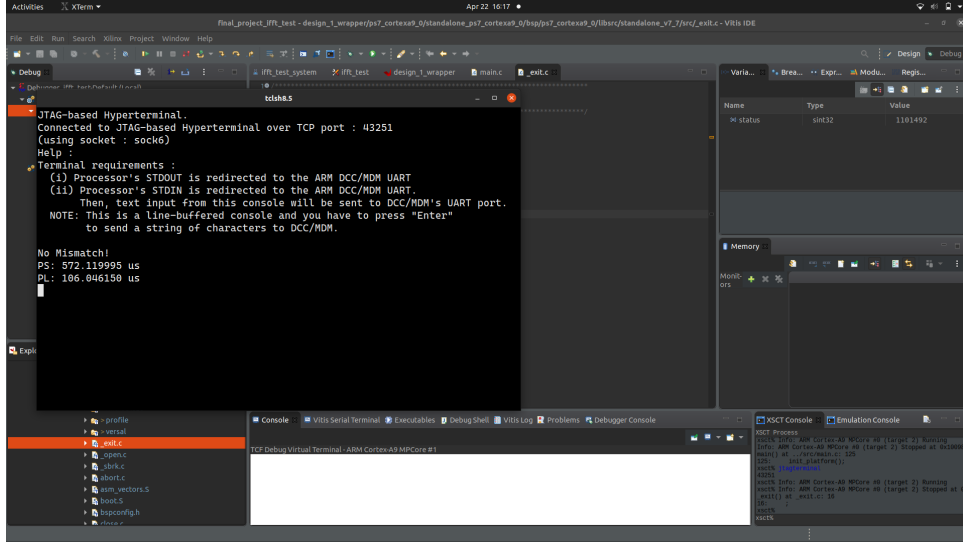


Figure 5: IFFT Performance With Precomputation (5.4 Speedup)

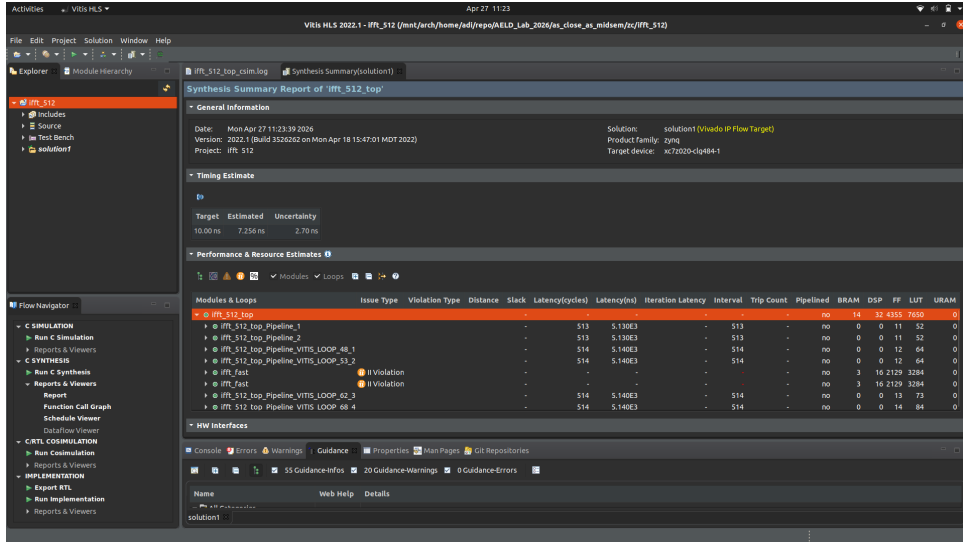


Figure 6: IFFT loop latency results

6. 1024-point FFT IP

6.1 Code Architecture

The top level function of the design is `fft_1024_top`. This function uses AXI4-Stream interfaces for both input and output data. The use of AXI streams allows continuous and standard data transfer between hardware IP blocks.

First, the function reads two arrays of 1024 complex float samples from the input stream and stores them into two local memory buffers, `ga` and `gb`. These correspond to the two Golay complementary sequences that are processed independently.

After storing the data, the top function calls the core processing block named `fft_fast` on both arrays. The `fft_fast` function performs an in-place Radix-2 Fast Fourier Transform. The

architecture of this core function is divided into two main stages:

- **Bit Reversal:** The input array is reordered using a predefined bit-reversal lookup table. If the reversed index is greater than the current index, the elements in the array are swapped.
- **Butterfly Stages:** The code loops through the ten stages of the FFT. At each stage, it computes complex multiplication using precomputed twiddle factors and performs the addition and subtraction operations required for the standard butterfly computation.

Once processing is complete, the top function writes both transformed arrays back to the output stream. The `last` signal is asserted on the final sample to indicate the end of packet transmission.

6.2 Optimizations and Results

In the first iteration of the design, the 1024-point FFT was implemented as a baseline hardware block without explicit HLS pragmas such as pipelining, array partitioning, or word-length optimization. This version was developed primarily to validate functional correctness and establish a synthesis baseline for later refinement.

To improve efficiency, precomputed lookup tables were used for both bit-reversal indexing and twiddle-factor storage. This avoids repeated runtime computation of twiddle coefficients and significantly simplifies butterfly-stage execution in hardware.

Synthesis of the design successfully met timing on the XC7Z020 target device, achieving an estimated clock period of 7.256 ns against a 10 ns target. Resource utilization for the synthesized IP is summarized below:

Table 2: 1024-point FFT IP synthesis summary

Resource	Utilization
BRAM	14
DSP	32
FF	3577
LUT	6964

Co-simulation of the generated RTL passed successfully, confirming that the hardware output matched the software reference implementation for both input sequences.

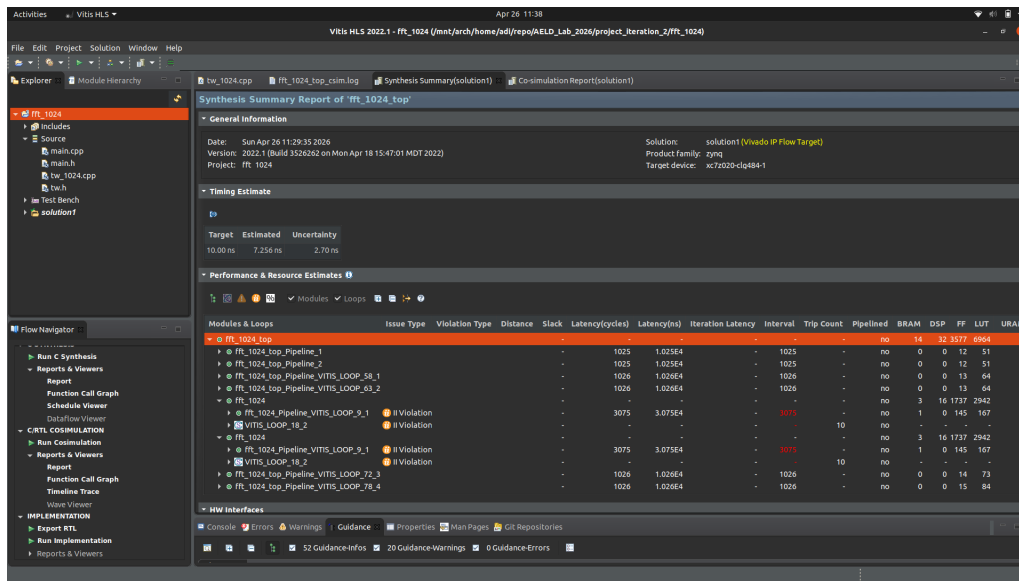


Figure 7: Loop latency and initiation interval

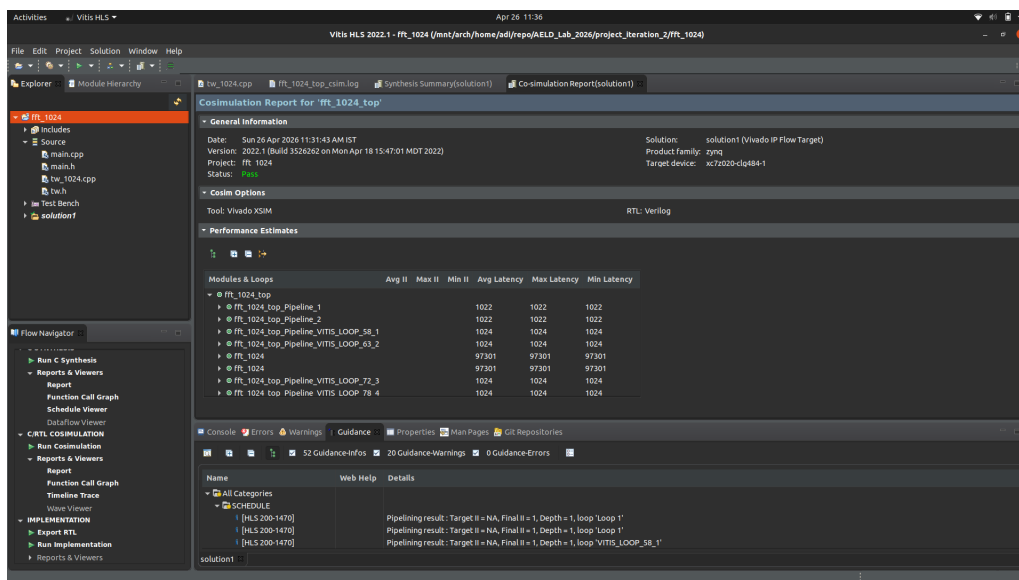


Figure 8: Loop latency and initiation interval

A further optimization was performed by inlining helper functions into the top-level compute path. This removed function-call overhead and exposed more logic directly to the HLS scheduler, allowing better loop scheduling and reduced overall latency in the FFT pipeline.

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration	Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
fft_1024_top			-	-	-	-	-	-	-	-	no	37	16	4627	8984	0
fft_1024_top_Pipeline_1			-	-	1025	1.02564	-	1025	-	-	no	0	0	12	51	0
Loop 1			-	-	1023	1.02364	1	1	1023	yes	-	-	-	-	-	-
fft_1024_top_Pipeline_2			-	-	1025	1.02564	-	1025	-	-	no	0	0	12	51	0
Loop 1			-	-	1023	1.02364	1	1	1023	yes	-	-	-	-	-	-
fft_1024_top_Pipeline_RA			-	-	1026	1.02664	-	1026	-	-	no	0	0	13	64	0
RA			-	-	1024	1.02464	1	1	1024	yes	-	-	-	-	-	-
fft_1024_top_Pipeline_RB			-	-	1026	1.02664	-	1026	-	-	no	0	0	13	64	0
RB			-	-	1024	1.02464	1	1	1024	yes	-	-	-	-	-	-
fft_1024_top_Pipeline_VITIS_LOOP_9_1	II Violation		-	-	2054	2.05464	-	2054	-	-	no	0	0	944	777	0
VITIS_LOOP_9_1	II Violation	Resource Limitation	-	-	2052	2.05264	7	2	1024	yes	-	-	-	-	-	-
fft_1024_top_Pipeline_VITIS_LOOP_9_11	II Violation		-	-	2054	2.05464	-	2054	-	-	no	0	0	944	777	0
VITIS_LOOP_9_11	II Violation	Resource Limitation	-	-	2052	2.05264	7	2	1024	yes	-	-	-	-	-	-
fft_1024_top_Pipeline_WA			-	-	1026	1.02664	-	1026	-	-	no	0	0	16	113	0
WA			-	-	1024	1.02464	2	1	1024	yes	-	-	-	-	-	-
fft_1024_top_Pipeline_WB			-	-	1026	1.02664	-	1026	-	-	no	0	0	17	124	0
WB			-	-	1024	1.02464	2	1	1024	yes	-	-	-	-	-	-
VITIS_LOOP_26_2			-	-	-	-	-	-	-	10	no	-	-	-	-	-
VITIS_LOOP_31_3			-	-	-	-	9221	-	-	-	no	-	-	-	-	-
fft_1024_top_Pipeline_VITIS_LOOP_33_4	II Violation		-	-	9218	9.21864	-	9218	-	-	no	0	0	620	591	0
VITIS_LOOP_33_4			-	-	9216	9.21664	18	18	512	yes	-	-	-	-	-	-
VITIS_LOOP_26_2			-	-	-	-	-	-	-	10	no	-	-	-	-	-
VITIS_LOOP_31_3			-	-	-	-	9221	-	-	-	no	-	-	-	-	-
fft_1024_top_Pipeline_VITIS_LOOP_33_42	II Violation		-	-	9218	9.21864	-	9218	-	-	no	0	0	620	591	0
VITIS_LOOP_33_4			-	-	9216	9.21664	18	18	512	yes	-	-	-	-	-	-

Figure 9: Loop latency after inlining functions

7. 1024-point IFFT IP

7.1 Inverse FFT Architecture

The top level function for the 1024-point Inverse Fast Fourier Transform is `ifft_1024_top`. Similar to the forward FFT IP, it uses AXI4 Stream interfaces for both input and output data transfer. The function reads two sets of 1024 complex samples from the input stream into two local arrays named `ga` and `gb`. These two arrays correspond to the two independently processed Golay paths.

The data is then passed to the core function named `ifft_fast`. The architecture of this function is very similar to the forward FFT. It first performs bit reversal using a predefined lookup table, followed by iterative butterfly computation across ten FFT stages using precomputed twiddle factors. The main difference in the inverse FFT is the normalization stage. After all butterfly stages are completed, every element in the array is multiplied by the inverse of the transform length, which in this case is $1/1024$.

Once both inverse transforms are completed, the top level function performs an additional summation step. Instead of streaming `ga` and `gb` separately, the corresponding output samples are added together and written as a single combined output stream. This directly implements the complementary summation property used in the Golay matched filtering pipeline.

Finally, the summed output is streamed back through the AXI interface, and the `last` signal is asserted on the final sample to indicate the end of packet transmission.

Module & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
ifft_1024_top			-	-	-	-	-	-	-	-	-	-	-	-	-
ifft_1024_top_Pipeline_1			-	-	1025	1.025E4	-	1025	-	no	18	36	4997	8301	0
Loop 1			-	-	1023	1.023E4	1	1	1023	yes	-	-	-	-	-
ifft_1024_top_Pipeline_2			-	-	1025	1.025E4	-	1025	-	no	0	0	12	51	0
Loop 1			-	-	1023	1.023E4	1	1	1023	yes	-	-	-	-	-
ifft_1024_top_Pipeline_VITIS_LOOP_48_1			-	-	1026	1.026E4	-	1026	-	no	0	0	13	64	0
VITIS_LOOP_48_1			-	-	1024	1.024E4	1	1	1024	yes	-	-	-	-	-
ifft_1024_top_Pipeline_VITIS_LOOP_53_2			-	-	1026	1.026E4	-	1026	-	no	0	0	13	64	0
VITIS_LOOP_53_2			-	-	1024	1.024E4	1	1	1024	yes	-	-	-	-	-
ifft_fast			-	-	-	-	-	-	-	no	5	16	2150	3285	0
ifft_fast_Pipeline_VITIS_LOOP_7_1	II Violation		-	-	3075	3.075E4	-	3075	-	no	1	0	145	167	0
VITIS_LOOP_7_1	II Violation Memory Dependency 1		-	-	3073	3.073E4	4	3	1024	yes	-	-	-	-	-
ifft_fast_Pipeline_VITIS_LOOP_33_5			-	-	1031	1.031E4	-	1031	-	no	0	0	300	126	0
VITIS_LOOP_33_5			-	-	1029	1.029E4	7	1	1024	yes	-	-	-	-	-
VITIS_LOOP_18_2			-	-	-	-	-	-	10	no	-	-	-	-	-
VITIS_LOOP_21_3			-	-	-	-	-	9221	-	no	-	-	-	-	-
ifft_fast_Pipeline_VITIS_LOOP_22_4	II Violation		-	-	9218	9.218E4	-	9218	-	no	4	10	1213	1745	0
VITIS_LOOP_22_4			-	-	9216	9.216E4	18	18	512	yes	-	-	-	-	-
ifft_fast			-	-	-	-	-	-	-	no	5	16	2150	3285	0
ifft_fast_Pipeline_VITIS_LOOP_7_1	II Violation		-	-	3075	3.075E4	-	3075	-	no	1	0	145	167	0
VITIS_LOOP_7_1	II Violation Memory Dependency 1		-	-	3073	3.073E4	4	3	1024	yes	-	-	-	-	-
ifft_fast_Pipeline_VITIS_LOOP_33_5			-	-	1031	1.031E4	-	1031	-	no	0	0	300	126	0
VITIS_LOOP_33_5			-	-	1029	1.029E4	7	1	1024	yes	-	-	-	-	-
VITIS_LOOP_18_2			-	-	-	-	-	-	10	no	-	-	-	-	-
VITIS_LOOP_21_3			-	-	-	-	-	9221	-	no	-	-	-	-	-
ifft_fast_Pipeline_VITIS_LOOP_22_4	II Violation		-	-	9218	9.218E4	-	9218	-	no	4	10	1213	1745	0
VITIS_LOOP_22_4			-	-	9216	9.216E4	18	18	512	yes	-	-	-	-	-
ifft_1024_top_Pipeline_VITIS_LOOP_85_3			-	-	1031	1.031E4	-	1031	-	no	0	4	627	896	0
VITIS_LOOP_85_3			-	-	1029	1.029E4	7	1	1024	yes	-	-	-	-	-

Figure 10: Vitis HLS implementation of 1024-point IFFT IP

7.2 IFFT Optimizations and Results

The 1024-point IFFT was further optimized using several HLS directives to improve throughput and reduce loop latency. First, the core function `ifft_fast` was marked with an inline pragma. This removes function-call overhead and exposes the complete computation path directly to the HLS scheduler, for optimization of the FFT stages.

Loop pipelining was introduced across the major computation stages. The bit-reversal loop, input parsing loops, normalization loop, and output writeback loop were all pipelined with an initiation interval of 1. This allows the hardware to process one sample every clock cycle in these stages, significantly improving overall throughput.

To improve memory access parallelism, cyclic array partitioning was applied to the internal buffers `x`, `ga`, and `gb` with a partition factor of 4. This reduces memory access bottlenecks during butterfly computation by allowing multiple concurrent accesses to different array partitions.

Additionally, a dependence pragma was introduced in the bit-reversal stage to inform the HLS compiler that inter-iteration dependencies do not exist. This enables more aggressive scheduling of the swap operations and improves pipeline efficiency.

With these optimizations, the synthesized design achieved lower loop latency and improved scheduling compared to the baseline implementation, while maintaining functional correctness. Co-simulation against the software reference implementation confirmed correct inverse transformation, normalization, and complementary summation of the two Golay paths.

The image shows two screenshots from a synthesis tool. The top screenshot is titled 'Performance & Resource Estimates' and displays a table of metrics for various modules and loops. The bottom screenshot is titled 'Performance Estimates' and displays a table of average, maximum, and minimum latency for the same modules and loops.

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
ifft_1024_top															
ifft_1024_top_Pipeline_1					1025	1.025E4		1025		no	42	36	5355	12541	0
ifft_1024_top_Pipeline_2					1025	1.025E4		1025		no	0	0	12	51	0
ifft_1024_top_Pipeline_RA					1026	1.026E4		1026		no	0	0	13	64	0
ifft_1024_top_Pipeline_RB					1026	1.026E4		1026		no	0	0	13	64	0
ifft_fast	!! Violation									no	5	16	2328	4717	0
ifft_fast	!! Violation									no	5	16	2328	4717	0
ifft_1024_top_Pipeline_WAB					1031	1.031E4		1031		no	0	4	629	976	0

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
ifft_1024_top						
ifft_1024_top_Pipeline_1				1022	1022	1022
ifft_1024_top_Pipeline_2				1022	1022	1022
ifft_1024_top_Pipeline_RA				1024	1024	1024
ifft_1024_top_Pipeline_RB				1024	1024	1024
ifft_fast				106518	106518	106518
ifft_fast				106518	106518	106518
ifft_1024_top_Pipeline_WAB				1029	1029	1029

Figure 11: Loop latency results after applying inlining, pipelining, and array partitioning

8. Trigger IP

8.1 Trigger IP Architecture

The next block in our design is the `trigger_ip` module. The main purpose of this module is to control the flow of data and break a continuous data stream into distinct batches or packets.

The function uses standard AXI4 Stream interfaces for both the input data and the output data. For system control, it utilizes AXI4 Lite interfaces. These control ports allow the main processing system to send a synchronous reset signal and to read a status flag indicating when the trigger has finished processing.

The core logic of the function relies on static counters. It keeps track of the number of samples processed in the current batch and the total number of batches completed. For every data word read from the input stream, the module checks if it has reached the end of the predefined batch size. If it is the last sample, the module sets the `last` variable to 1 on the output word. This action generates the hardware TLAST signal.

Once the maximum number of batches is reached, the module asserts the `trigger_done` signal and stops processing new data. The function also uses a pipeline pragma to achieve an initiation interval of 1. This ensures the hardware can process one incoming data sample every single clock cycle.

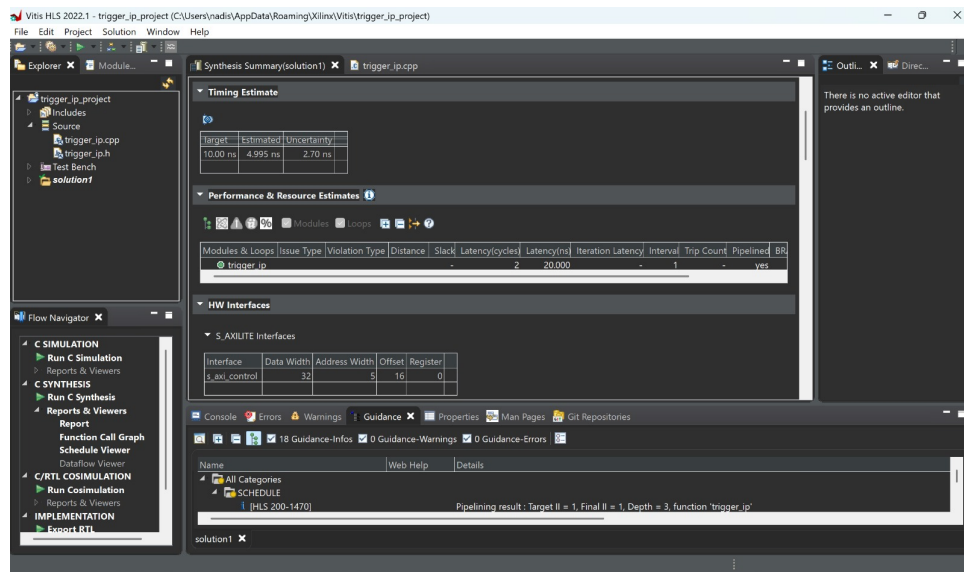


Figure 12: Vitis HLS Synthesis Report for Trigger IP

8.2 System Integration and Packet Mode

In the overall Vivado block design, the output of the `trigger_ip` connects directly to an AXI4 Stream Data FIFO. The processing system controls the IP blocks and uses a Direct Memory Access block to handle high speed memory transfers.

As shown in the configuration figure below, the AXI4 Stream Data FIFO is explicitly configured with packet mode enabled. When packet mode is turned on, the FIFO will store incoming data and will not release it to the DMA until it receives a TLAST signal indicating a complete packet. Because our custom trigger IP accurately generates this TLAST signal at the end of every batch, the downstream FIFO and DMA can safely and correctly transfer the grouped data into memory.

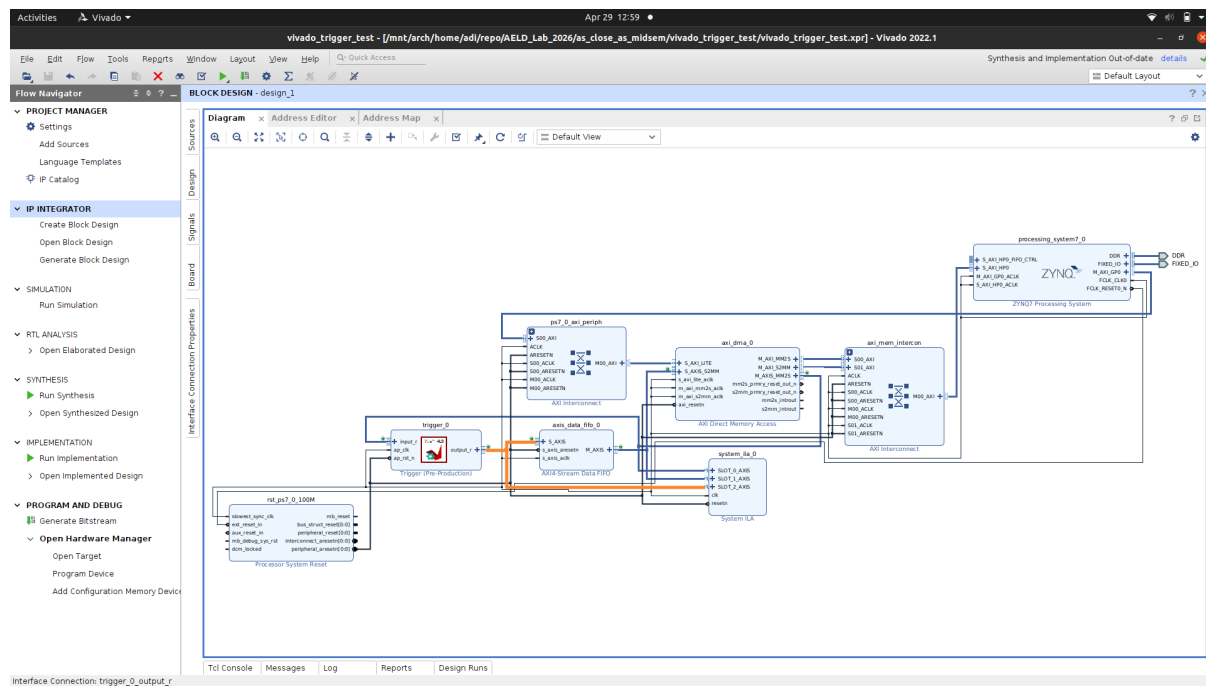


Figure 13: Vivado Block Design Showing Trigger IP Integration

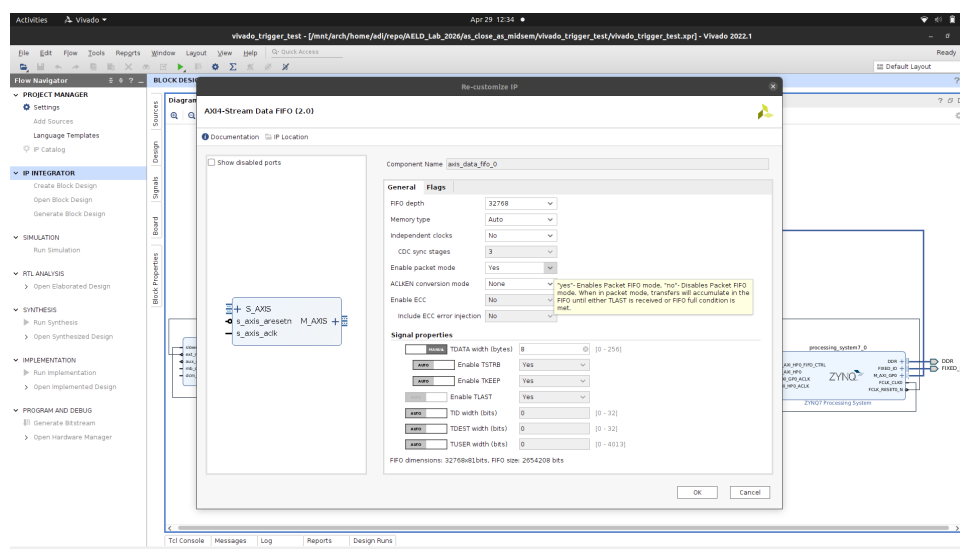


Figure 14: AXI4-Stream Data FIFO Configuration with Packet Mode Enabled

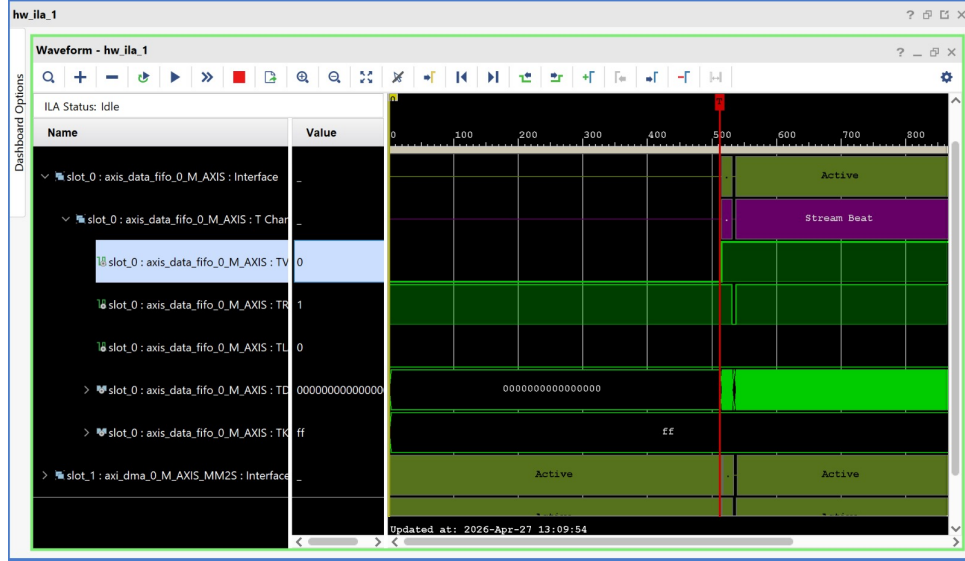


Figure 15: Trigger IP Functionality Verification with ILA

9. Doppler FFT IP

9.1 Doppler FFT IP Architecture

The Doppler FFT IP is used to convert the compressed range profile into Doppler frequency information. Its main purpose is to process slow-time radar samples and extract the phase progression caused by target motion, which is then used to estimate radial velocity. The function uses standard AXI4 Stream interfaces for both input and output data, allowing seamless integration with the other processing blocks in the hardware pipeline.

The top level function of the design is `doppler_fft_256`. This function reads 256 complex float samples from the input stream and stores them in a local memory buffer named `range_dop_map`. Once all samples are collected, the function performs two sequential processing stages.

The first stage is the FFT computation. This is handled by the core function `fft_fast`, which performs an in-place Radix-2 Cooley–Tukey Fast Fourier Transform over eight butterfly stages. Similar to the earlier FFT IP designs, the architecture is divided into two main steps:

- **Bit Reversal:** The input array is reordered using a precomputed bit-reversal lookup table. This avoids runtime bit manipulation and ensures that samples are arranged correctly for iterative butterfly computation.
- **Butterfly Stages:** The function loops through the FFT stages, computes complex multiplication using precomputed twiddle factors, and performs the addition and subtraction operations required for the standard butterfly calculation.

After the FFT stage is complete, the second processing stage performs an in-place FFT shift using the `fftshift_inplace` function. This swaps the two halves of the transformed output array, centering the zero-Doppler component in the spectrum. This step is important because it

rearranges the Doppler bins into the standard frequency ordering used for Range-Doppler map visualization and target detection.

Once processing is complete, the updated 256-point Doppler spectrum is written back to the output stream. The final output sample is marked with the `last` signal set to 1, generating the TLAST signal required for packet-based downstream transfer.

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
doppler_fft_256			-	-	-	-	-	-	-	no	7	16	1906	3796	0
doppler_fft_256_Pipeline_1			-	-	257	2.570E3	-	257	-	no	0	0	10	53	0
Loop 1			-	-	255	2.550E3	1	1	255	yes	-	-	-	-	-
doppler_fft_256_Pipeline_VITIS_LOOP_58_1			-	-	258	2.580E3	-	258	-	no	0	0	11	65	0
VITIS_LOOP_58_1			-	-	256	2.560E3	1	1	256	yes	-	-	-	-	-
fft_fast			-	-	-	-	-	-	-	no	3	16	1812	3001	0
fft_fast_Pipeline_VITIS_LOOP_8_1	II Violation		-	-	771	7.710E3	-	771	-	no	1	0	131	168	0
VITIS_LOOP_8_1	II Violation	Memory Dependency 1	-	-	769	7.690E3	4	1	256	yes	-	-	-	-	-
VITIS_LOOP_19_2			-	-	-	-	-	-	8	no	-	-	-	-	-
VITIS_LOOP_22_3			-	-	-	-	2309	-	-	no	-	-	-	-	-
fft_fast_Pipeline_VITIS_LOOP_23_4	II Violation		-	-	2306	2.306E4	-	2306	-	no	2	16	1489	2395	0
VITIS_LOOP_23_4			-	-	2304	2.304E4	18	18	128	yes	-	-	-	-	-
doppler_fft_256_Pipeline_VITIS_LOOP_38_1	II Violation		-	-	258	2.580E3	-	258	-	no	0	0	43	132	0
VITIS_LOOP_38_1	II Violation	Resource Limitation	-	-	256	2.560E3	2	2	128	yes	-	-	-	-	-
doppler_fft_256_Pipeline_VITIS_LOOP_66_2			-	-	258	2.580E3	-	258	-	no	0	0	13	85	0
VITIS_LOOP_66_2			-	-	256	2.560E3	2	1	256	yes	-	-	-	-	-

Figure 16: Vitis HLS implementation of Doppler FFT IP

9.2 Verification and Results

To verify correctness, the hardware IP output was compared against a software reference implementation using the same FFT and FFT-shift operations. The software model processed the correlation output and generated a reference Doppler spectrum, while the HLS IP processed the same input stream through AXI4 Stream interfaces.

The outputs from both implementations were compared sample-by-sample. A tolerance threshold was used to account for floating-point numerical variation, and the generated hardware output matched the software reference successfully.

This verification confirmed that the Doppler FFT IP correctly performs spectral transformation and frequency centering, making it suitable for direct integration into the complete radar processing chain.

10. Complex Multiplication IP

10.1 Multiplication IP Architecture

The next block in our design is the complex multiplication IP module. The main purpose of this module is to perform element-wise multiplication of the FFT output with the precomputed complex conjugate reference sequences. This operation forms the core multiplication stage of frequency-domain matched filtering.

The function uses standard AXI4 Stream interfaces for both input data and output data, allowing direct integration with the FFT and IFFT processing blocks in the hardware pipeline.

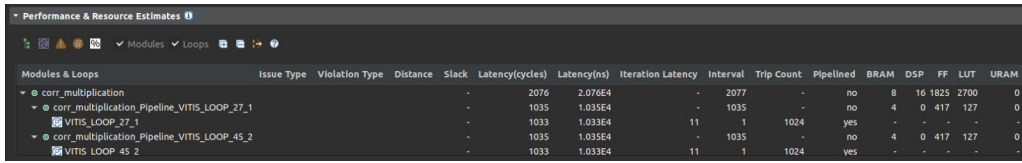
The top level function of the design is `corr_multiplication`. This function processes two sequential sets of 1024 complex samples, corresponding to the transformed Golay sequences G_a and G_b . The architecture handles these two paths independently and processes them one after

the other.

For the first input sequence, the module reads each sample from the input stream and stores it in a local array named `ga`. It then performs element-wise complex multiplication with the precomputed conjugate reference array `ga_conj`. The resulting multiplied samples are stored in `output_ga` and immediately streamed back to the output interface.

The same process is then repeated for the second input sequence. The incoming samples are stored in the local array `gb`, multiplied element-wise with the conjugate reference array `gb_conj`, and written back through the output stream as `output_gb`.

For both processing stages, the final sample is marked by setting the `last` signal to 1. This generates the AXI TLAST signal, ensuring proper packet termination for downstream modules.



Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
corr_multiplication	-	-	-	-	2076	2.076E4	-	2077	-	no	8	16	1825	2700	0
corr_multiplication_Pipeline_VITIS_LOOP_27_1	-	-	-	-	1035	1.035E4	-	1035	-	no	4	0	417	127	0
VITIS_LOOP_27_1	-	-	-	-	1033	1.033E4	11	1	1024	yes	-	-	-	-	-
corr_multiplication_Pipeline_VITIS_LOOP_45_2	-	-	-	-	1035	1.035E4	-	1035	-	no	4	0	417	127	0
VITIS_LOOP_45_2	-	-	-	-	1033	1.033E4	11	1	1024	yes	-	-	-	-	-

Figure 17: Vitis HLS implementation of complex multiplication IP

10.2 Optimizations, Verification and Results

The complex multiplication IP was optimized using HLS directives to improve throughput and memory access efficiency. Cyclic array partitioning with a factor of 4 was applied to the pre-computed conjugate lookup arrays `ga_conj` and `gb_conj`. This allows parallel memory accesses during multiplication and reduces memory bottlenecks while reading reference coefficients.

In addition, both the G_a and G_b processing loops were pipelined with an initiation interval of 1. This allows the hardware to continuously read one sample, perform one complex multiplication, and write one output sample every clock cycle. As a result, the multiplication stage operates as a high-throughput streaming compute block with minimal processing overhead.

To verify correctness, the hardware output was compared against a software reference implementation using the same precomputed conjugate reference arrays. The generated RTL output matched the software output successfully for both G_a and G_b , confirming correct element-wise multiplication, proper packet termination through TLAST generation, and correct sequential processing of both Golay paths.

These optimizations improved throughput while maintaining functional correctness, making the multiplication IP an efficient middle stage between the forward FFT and inverse FFT blocks in the complete correlation pipeline.

11. Correlation Pipeline (FFT-Multiplication-IFFT)

11.1 IP Integration

The correlation stage combines the previously developed FFT, complex multiplication, and IFFT IP blocks into a complete hardware matched-filtering pipeline. Instead of implementing

The input stream first enters the forward FFT IP, which transforms the incoming time-domain sequences into the frequency domain. These transformed outputs are then passed directly into the complex multiplication IP, where element-wise multiplication is performed with the pre-computed conjugate reference sequences. The multiplied frequency-domain outputs are then streamed into the IFFT IP, which performs inverse transformation and complementary summation of the G_a and G_b paths to generate the final compressed range profile.

Because all three stages use standard AXI4 Stream interfaces, data flows continuously between IP blocks without intermediate software intervention. The correct propagation of packet boundaries through TLAST signaling was also preserved across all stages, allowing reliable packetized transfer throughout the full correlation chain.

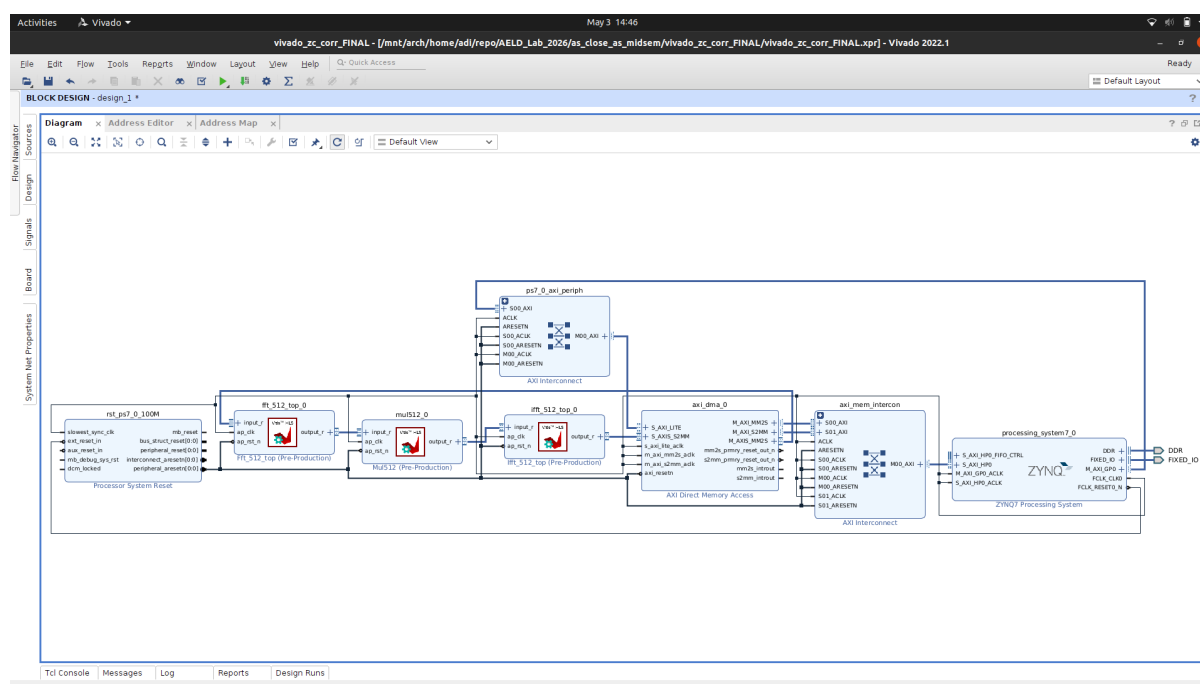


Figure 18: Vivado block design showing integration of FFT, multiplication, and IFFT IP blocks for hardware correlation

11.2 Testing and Results

After integration, the complete correlation pipeline was tested against a software reference implementation. The hardware output was compared against software-generated matched filtering results to verify numerical correctness across the full FFT–Multiply–IFFT chain.

Testing also verified correct sequential handling of the G_a and G_b paths, proper complementary summation in the final IFFT stage, and correct TLAST generation at packet boundaries. Since each individual IP had already been independently verified, correlation-stage testing primarily focused on end-to-end stream alignment, packet integrity, and correctness of chained execution

across the full hardware datapath.

The integrated hardware correlation pipeline successfully reproduced the software reference output, confirming correct operation of the complete matched filtering chain. This established correlation as a stable intermediate processing block, ready for direct integration with the Doppler FFT stage in the final radar architecture.

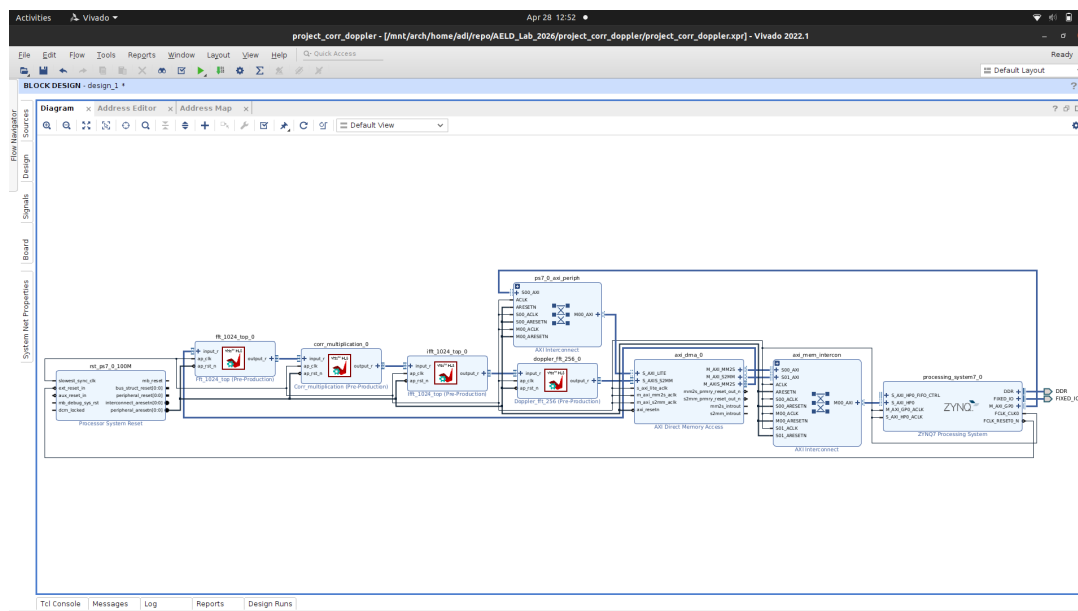


Figure 19: Testing and verification results for the complete hardware correlation pipeline

11.3 Correlation and Doppler Integration

Problem Faced

After integrating the FFT, complex multiplication, and IFFT IP blocks, the next step was to directly connect the correlation output to the Doppler FFT stage. However, an architectural mismatch was encountered.

The correlation block generates output row-wise, producing one compressed range profile of length 1024 for each pulse. Over 256 pulses, this forms a full correlation matrix of size 1024×256 . The Doppler FFT, however, operates column-wise, since each range bin must be observed across all 256 pulses before Doppler processing can be performed.

This means the complete correlation matrix must first be accumulated before it can be sent to the Doppler block.

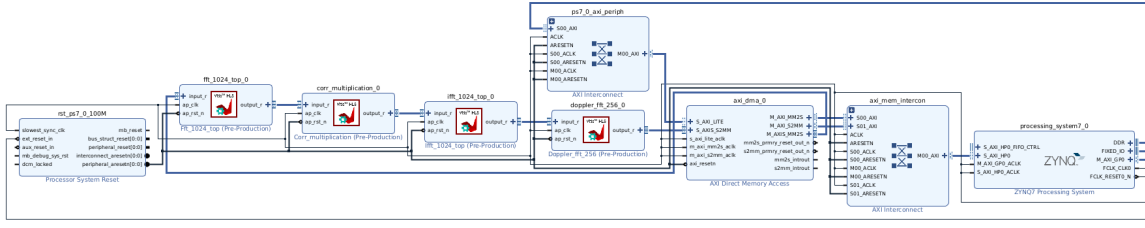


Figure 20: Initial correlation and Doppler hardware integration

Memory Bottleneck and Final Architecture

The full 1024×256 complex matrix is approximately 1 MB in size. Attempting to buffer this entirely inside PL BRAM resulted in heavy BRAM over-utilization, making a fully streamed PL-only architecture infeasible on the XC7Z020 device.

To solve this, the correlation output is first written back to PS DDR memory using AXI DMA. Once the full matrix is accumulated in DDR, it is streamed back column-wise into the Doppler FFT IP for processing. This removes the BRAM bottleneck, but introduces an additional memory loopback through DDR, which becomes the main throughput limitation in the current system.

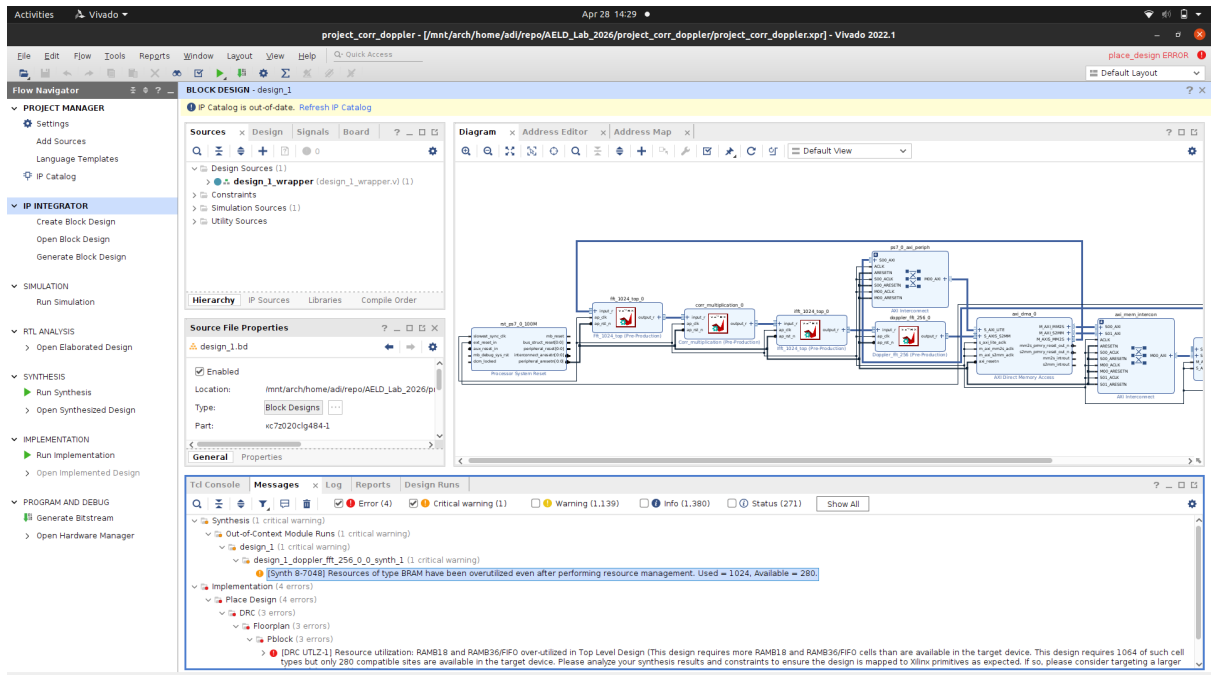


Figure 21: Final architecture using PS DDR loopback for matrix buffering

12. Complete Zadoff Chu Radar Hardware Design

12.1 Block Design

The hardware architecture for the Zadoff Chu radar was developed using a progressive approach. First, we implemented and tested the isolated correlation chain. This initial test design uses a single Direct Memory Access block to feed data through the forward FFT, the complex multiplier, and the inverse FFT blocks.

Once the correlation chain was verified, we built the complete system architecture. The full block design integrates the Doppler FFT processing block and our custom trigger IP. To manage the different data rates and prevent memory bottlenecks, this complete system uses three independent DMA blocks operating in parallel.

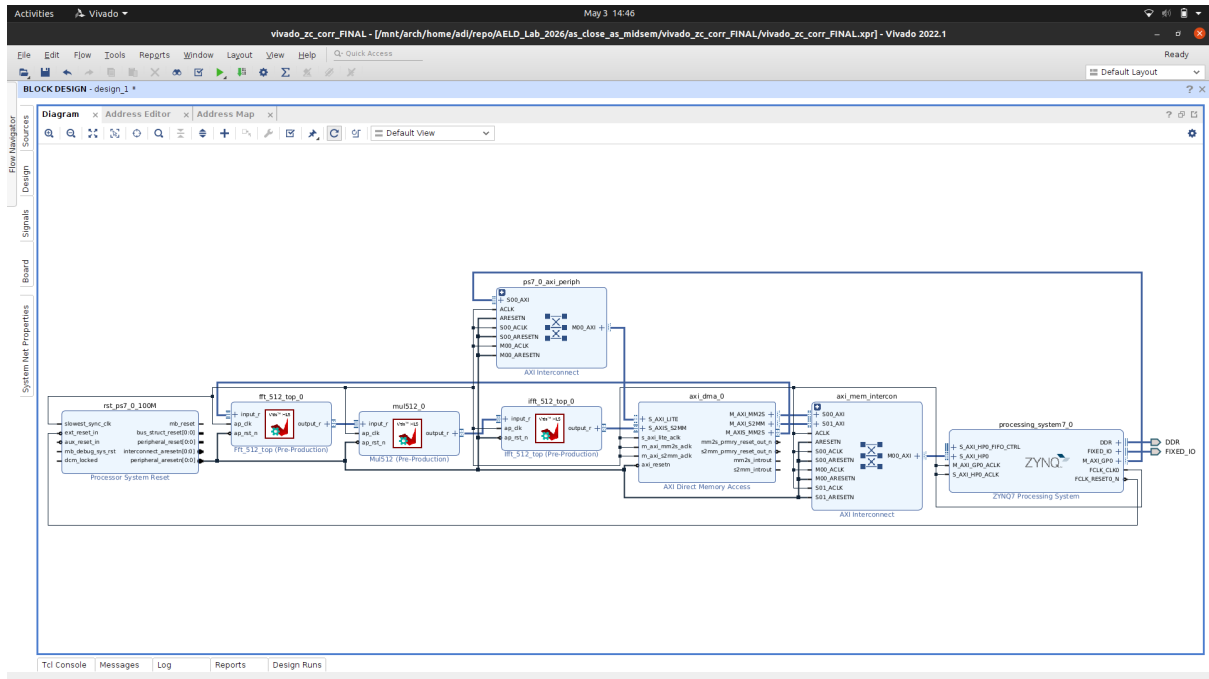


Figure 22: Basic Zadoff Chu Correlation Block Design with Single DMA

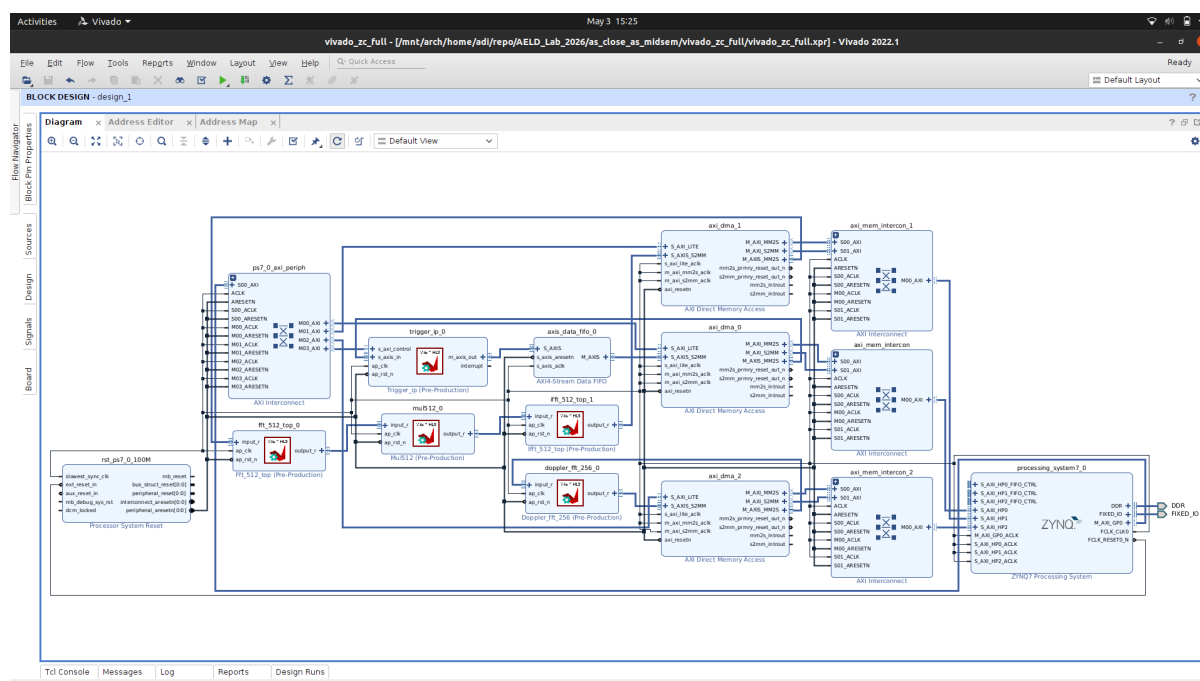


Figure 23: Complete Zadoff Chu Radar Vivado Block Design

Because the Zadoff Chu sequence length is 512, the memory requirements were significantly lower than our previous designs. The post implementation resource utilization report shows a very comfortable BRAM usage of exactly 68 percent.

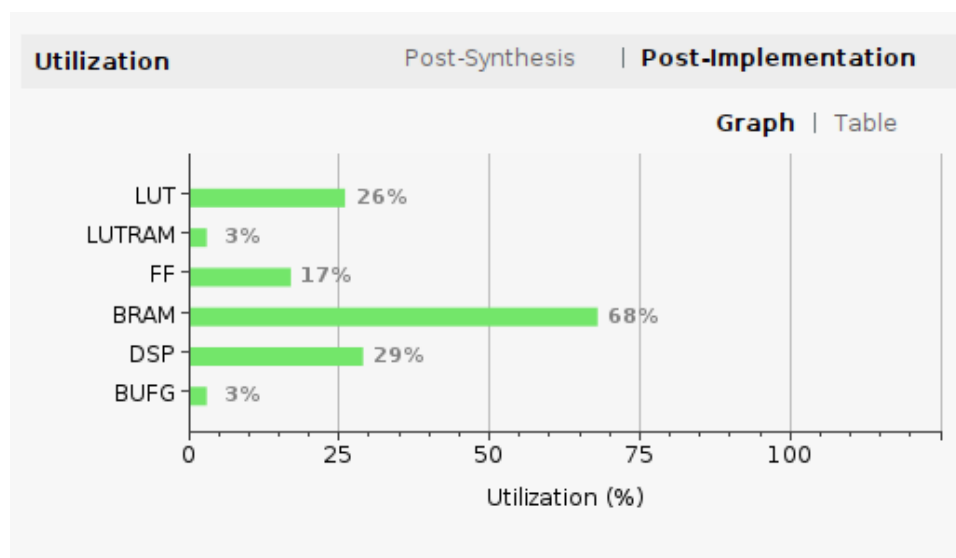


Figure 24: Optimized Post Implementation Resource Utilization for Zadoff Chu

12.2 Dataflow

The processing dataflow is fully accelerated in hardware. The incoming raw radar samples are passed into the 512 point forward FFT block. The output in the frequency domain goes directly into the multiplication block where it is multiplied with the precomputed Zadoff Chu reference signal. The result is then passed through the inverse FFT block to produce the final

time domain correlation array for range detection.

At the same time, the separate Doppler FFT block processes the samples across multiple pulses to calculate the target velocity. The custom trigger IP precisely batches the data streams to ensure the AXI4 Stream FIFOs and the DMAs correctly route the packets to the processing system memory.

12.3 Output Validation

To validate the complete Zadoff Chu hardware design, we ran an extensive Monte Carlo simulation consisting of 50 full runs. The terminal output confirms that the system processes the complex radar algorithms with high stability and speed. For this sequence length, the hardware achieved an average correlation time of approximately 0.24 seconds and a Doppler processing time of approximately 0.11 seconds.

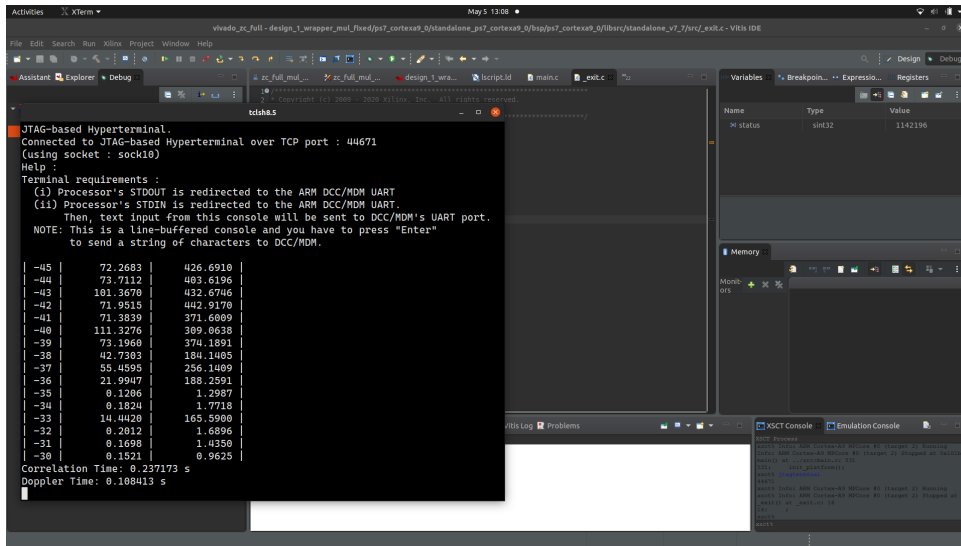


Figure 25: Results of 50 Monte Carlo Runs for the Zadoff Chu Hardware

13. Complete Golay Radar Hardware Design

13.1 Block Design

The hardware design for the complete Golay radar was built in stages to ensure each part worked correctly. First, we tested the basic correlation chain using a single Direct Memory Access block. This chain includes the forward FFT, the complex multiplication, and the inverse FFT blocks. Next, we added the Doppler FFT block to the design. To handle the data flow better, we upgraded the system to use two separate DMA blocks. Finally, we assembled the complete block design. The full design includes the custom trigger IP and three DMAs to manage the parallel data streams perfectly.

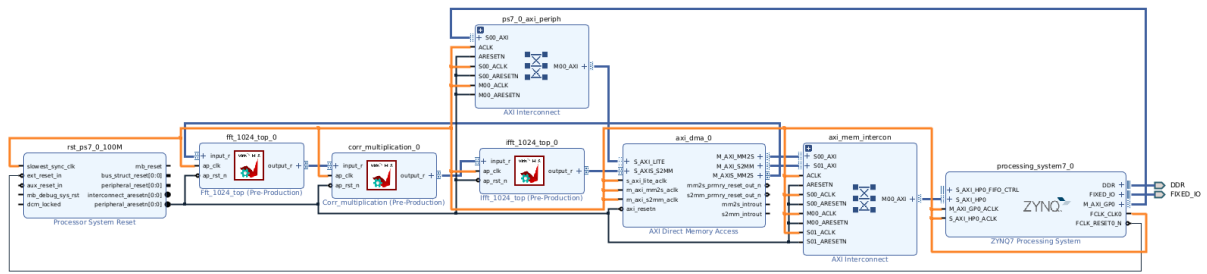


Figure 26: Basic Correlation Chain with Single DMA

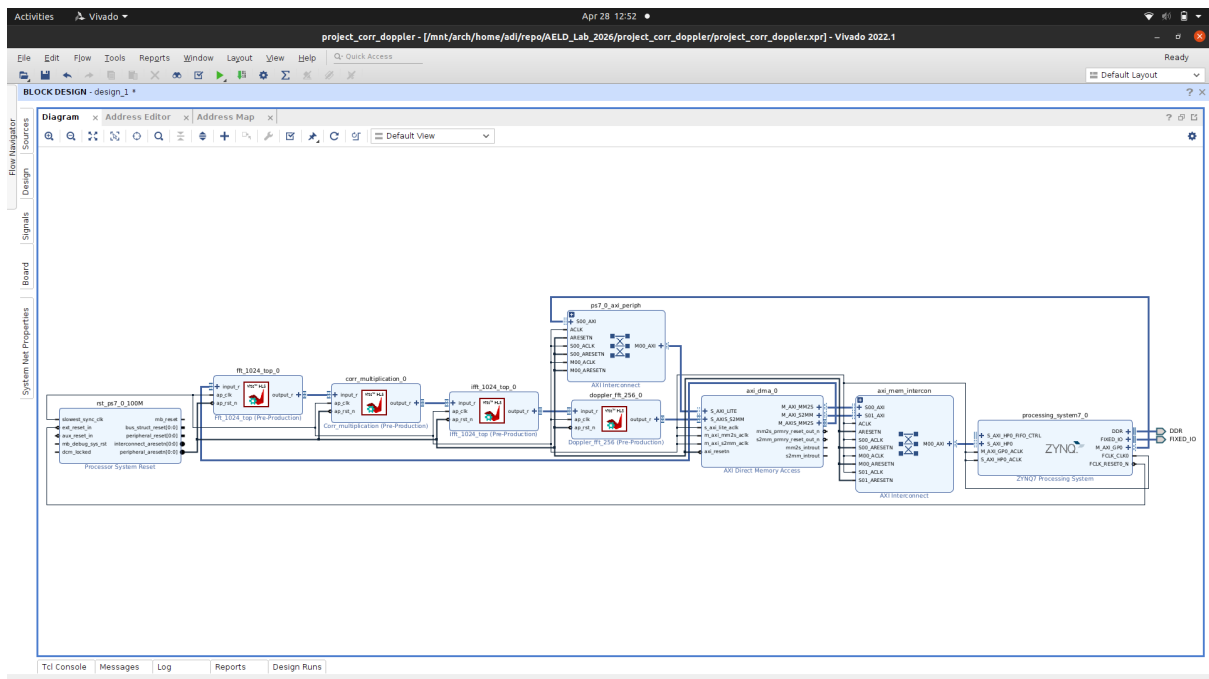


Figure 27: Correlation and Doppler Processing Block Design



27

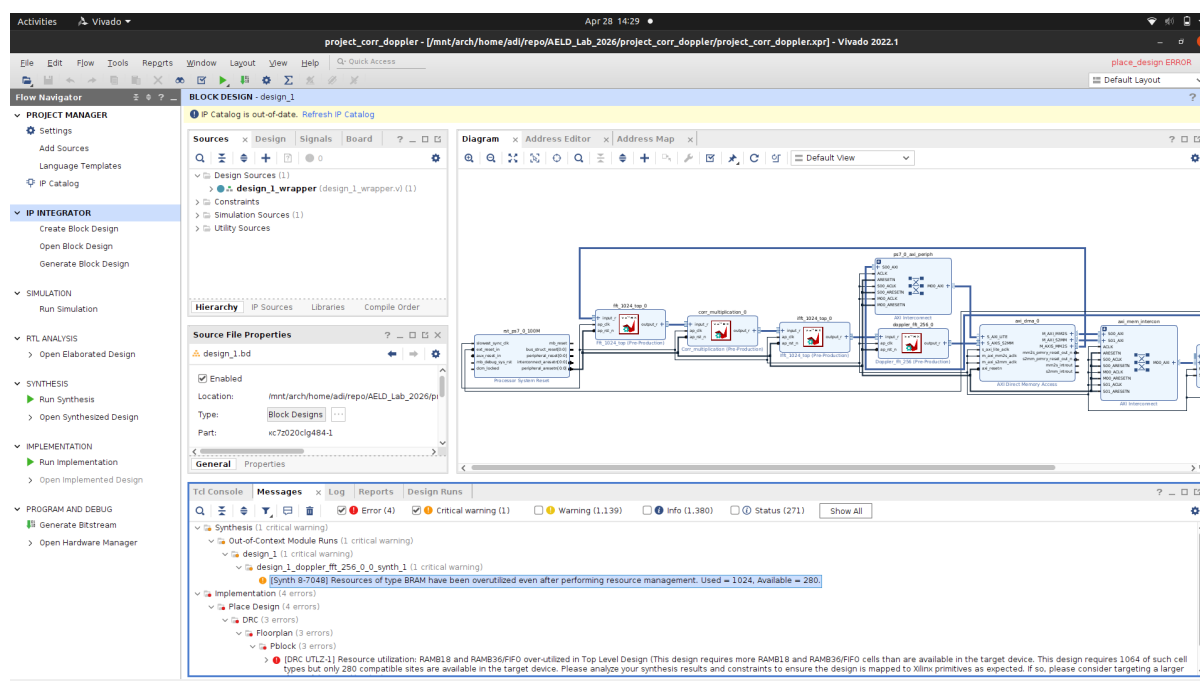


Figure 30: Initial Synthesis Error Showing BRAM Overutilization

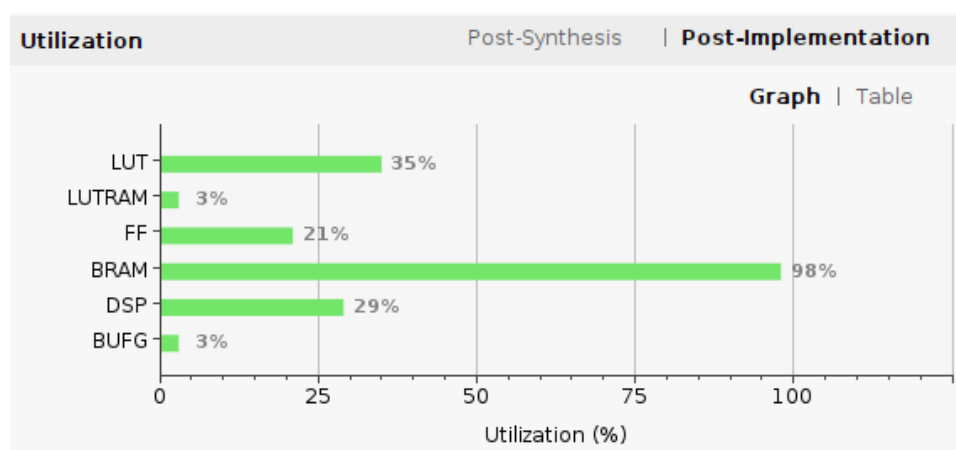


Figure 31: Optimized Post Implementation Resource Utilization

13.2 Dual-path Correlation Architecture

The correlation process is fully hardware accelerated in our design. The incoming radar signals are passed into the forward FFT block. The frequency domain output goes directly into the correlation multiplication block where it is multiplied with the reference signal. Finally, the data goes through the inverse FFT block to produce the final time domain correlation result.

13.3 Doppler Processing

For velocity detection, a separate Doppler FFT block processes the samples across multiple pulses. Because the correlation and Doppler operations have different data rates and sizes,

we dedicated independent DMA blocks to them. This architecture prevents bottlenecks in the system memory and allows the processor to read range and velocity data at the same time.

13.4 Output Validation

To prove the hardware works correctly, we created specific test vectors in the software. We set a true target range of 178 and a true target velocity of 39. The hardware output closely matched these inputs with a range estimate of 178.0 and a velocity estimate of 40.05.

```
// randomly choose range and velocity of the target
// range: [50, 200]
// velocity: [-100, 100]
/* float true_r = 50.0 + 150.0 * rng_float(); */
/* float true_v = -100.0 + 200.0 * rng_float(); */
float true_r = 178;
float true_v = 39;
```

Figure 32: Software Test Vectors for Range and Velocity

We also ran a full 50 iteration Monte Carlo simulation to test the stability of the entire system. The terminal output confirms successful execution. The total correlation time was roughly 0.55 seconds and the Doppler processing time was roughly 0.22 seconds.

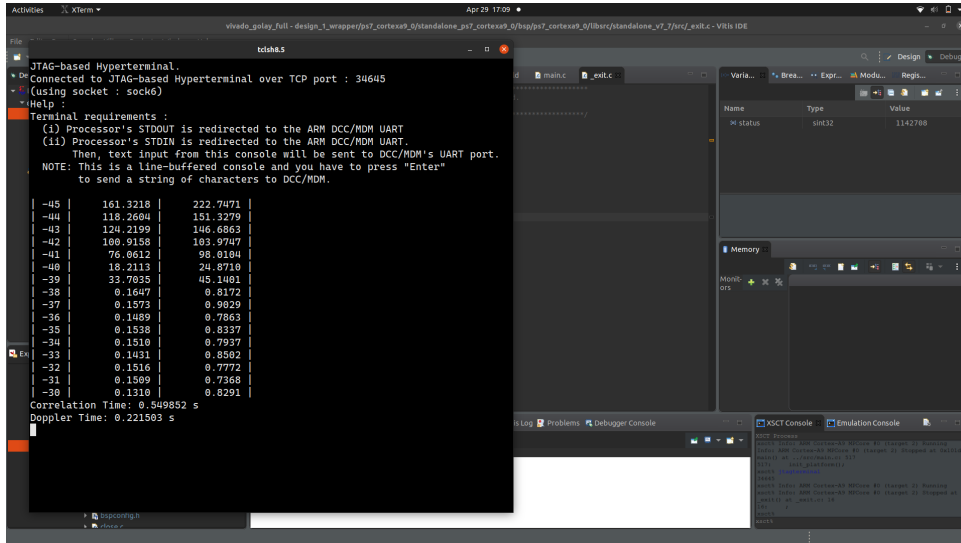


Figure 33: Results of 50 Monte Carlo Runs on the Hardware

Finally, we ported the design to a different evaluation board, the ZC706. This was done because ZC706 boasts more BRAM and DSP potentially increasing performance however since it operates at 50MHz, the performance degraded.

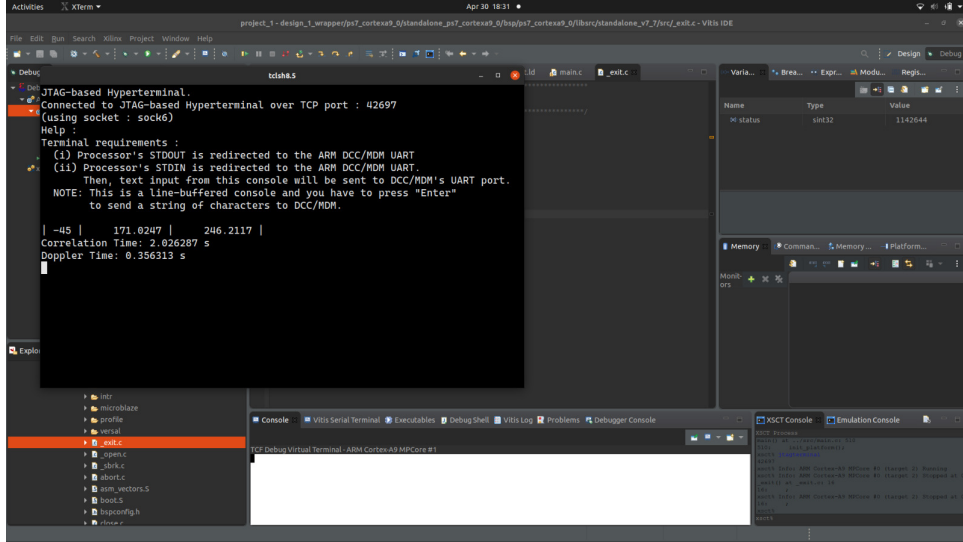


Figure 34: First Hardware Test on the ZC706 Evaluation Board

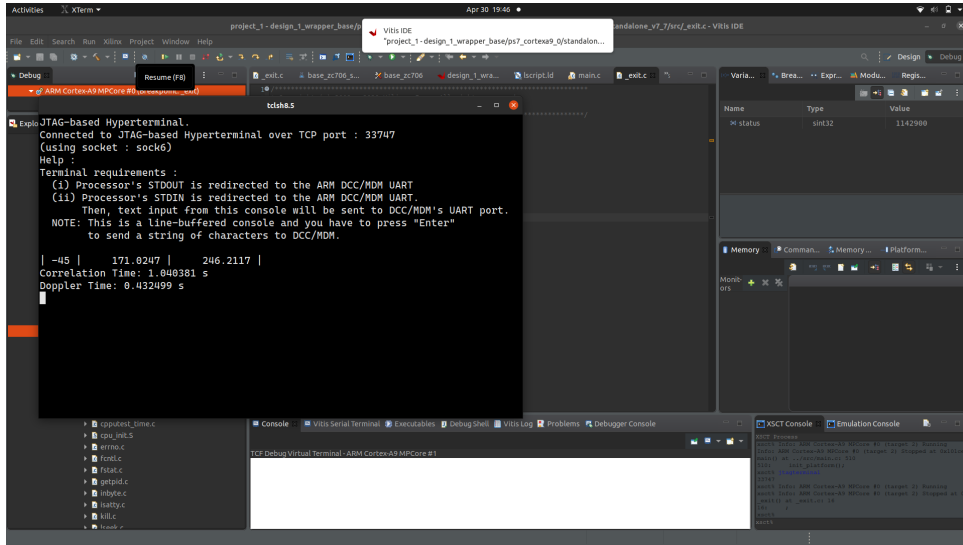


Figure 35: Second Hardware Test on the ZC706 Evaluation Board

14. Further Optimizations: Word-Length Optimization & Memory-Mapped Implementations

14.1 Memory Mapped Golay

We also explored a memory mapped approach for the Golay radar system. The custom logic blocks and the overall architecture for this design were successfully synthesized and verified using High Level Synthesis. This step confirms that the core processing algorithms and logic work correctly at the module level.

However, due to project time constraints, we could not fully integrate and verify the complete functional block design within the Vivado IDE environment. The figures below display the block design and the IP configuration for this memory mapped implementation.

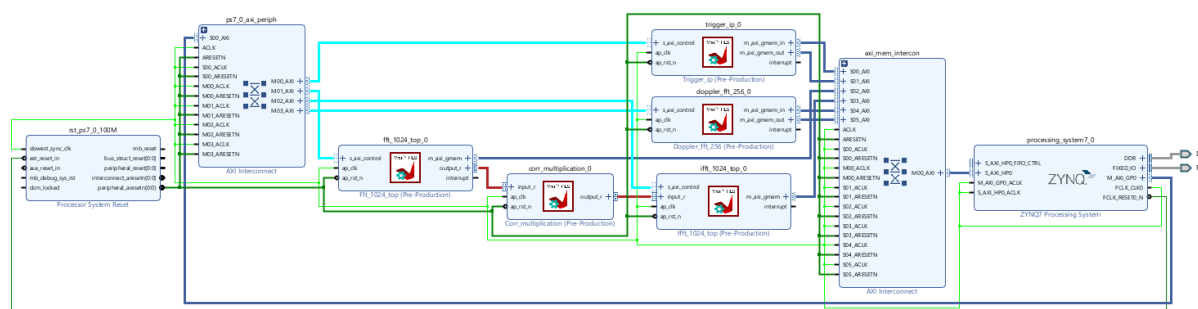


Figure 36: Memory Mapped Golay Block Design

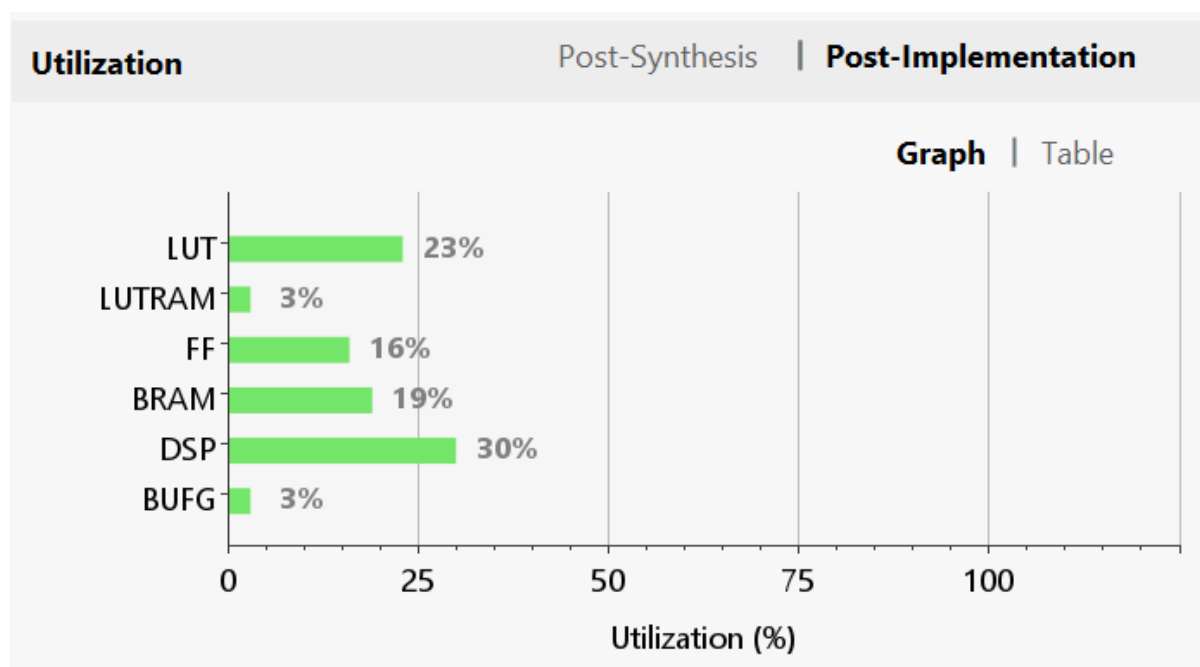


Figure 37: Utilisation for Memory Mapped Implementation

14.2 Memory Mapped Zadoff-Chu

Similar to the Golay approach, a memory mapped approach was also explored for ZC data.

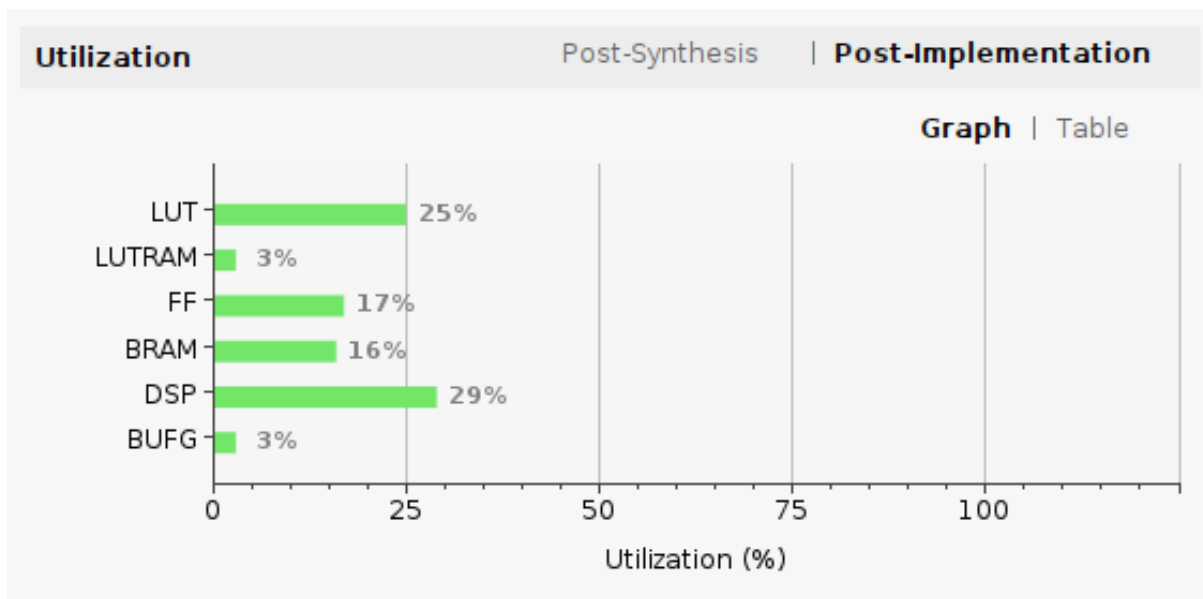


Figure 38: Utilisation for Memory Mapped Implementation

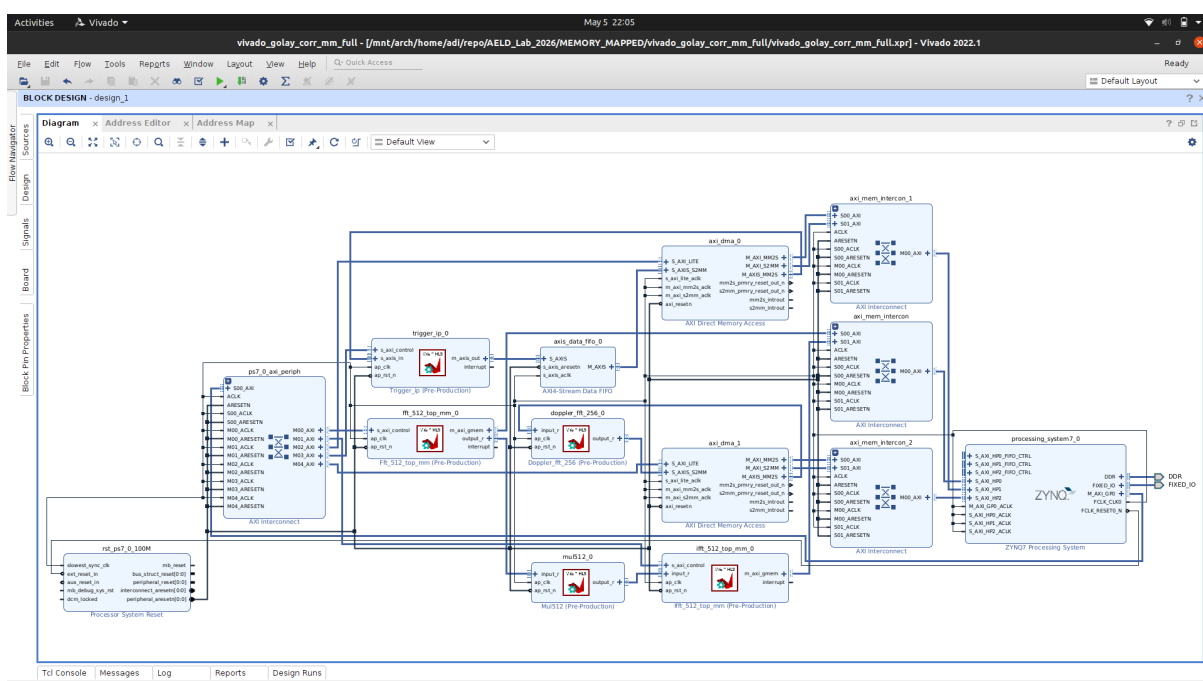


Figure 39: Block Diagram for Memory Mapped Implementation

The results for the same were obtained and are presented below.

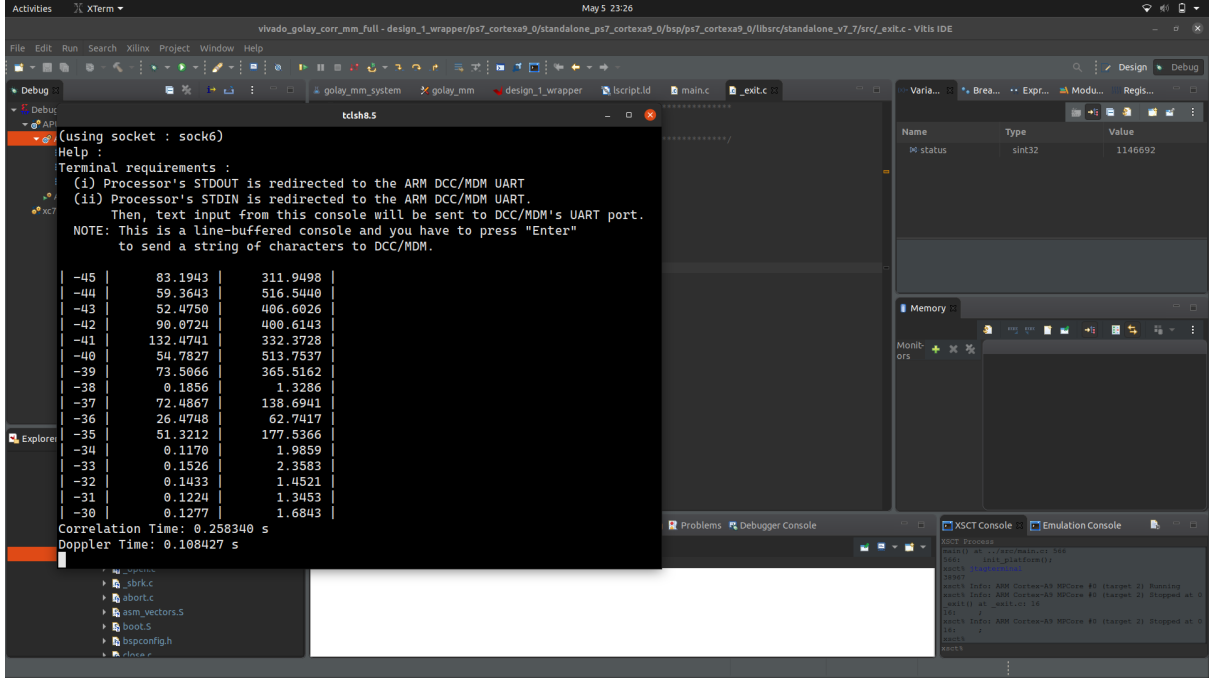


Figure 40: Results for Memory Mapped Implementation

14.3 Word Length Optimization

Iteration 1

To further improve the efficiency of our hardware design, we performed word length optimization. We transitioned the core processing blocks from standard floating point arithmetic to fixed point arithmetic. This was done using the arbitrary precision fixed point library in High Level Synthesis.

During the first iteration of this optimization, we carefully selected the bit widths for the different stages to balance mathematical precision and hardware resource usage. Based on the data ranges expected at each step, we configured the main modules as follows:

- **Forward FFT:** The main data array uses 32 total bits with 16 integer bits to prevent overflow during the mathematical calculations. The twiddle factors, which are always between negative one and positive one, use 18 total bits with only 2 integer bits.
- **Inverse FFT:** Because the data is scaled down before entering the inverse transform, the main array uses 32 total bits with only 2 integer bits. The twiddle factors use 24 total bits with 2 integer bits.
- **Multiplication Block:** The standard correlation data uses 32 total bits with 12 integer bits. The conjugate reference data requires slightly less precision and uses 27 total bits with 11 integer bits.

Implementing these fixed point data types significantly reduced the hardware footprint. The synthesis reports show a major drop in the required Block RAM when comparing the original floating point baseline to the newly optimized fixed point version.

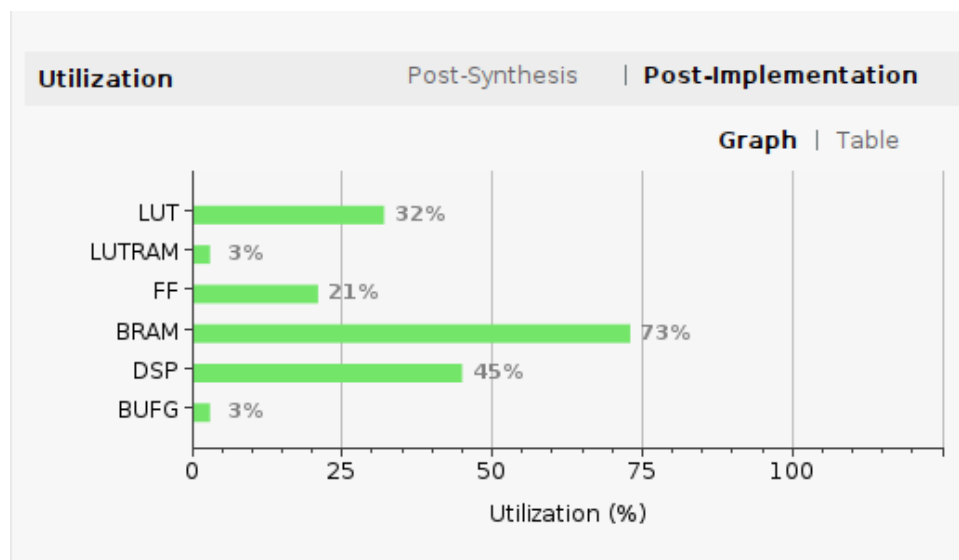


Figure 41: Resource Utilization Before Word Length Optimization

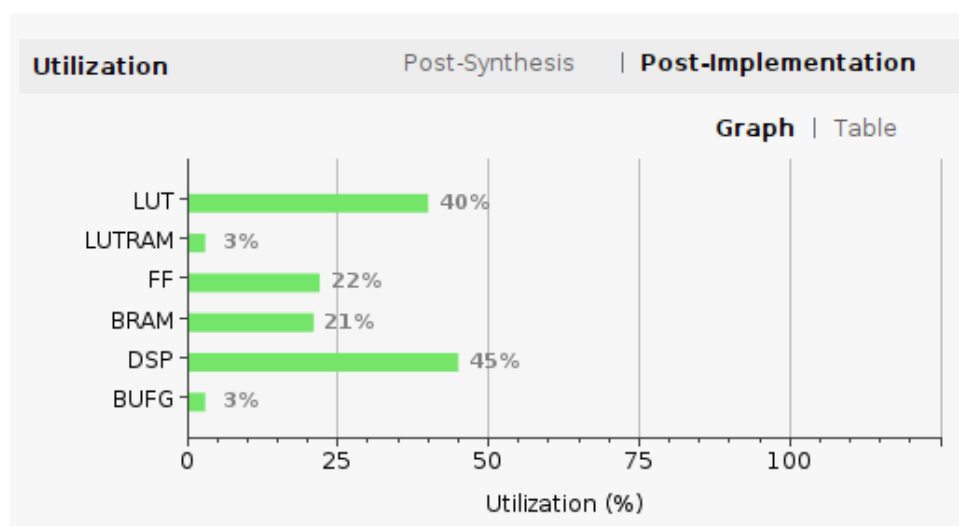


Figure 42: Resource Utilization After First Iteration of Word Length Optimization

We integrated these optimized fixed point modules into a complete Vivado block design to ensure the data streams connected properly.

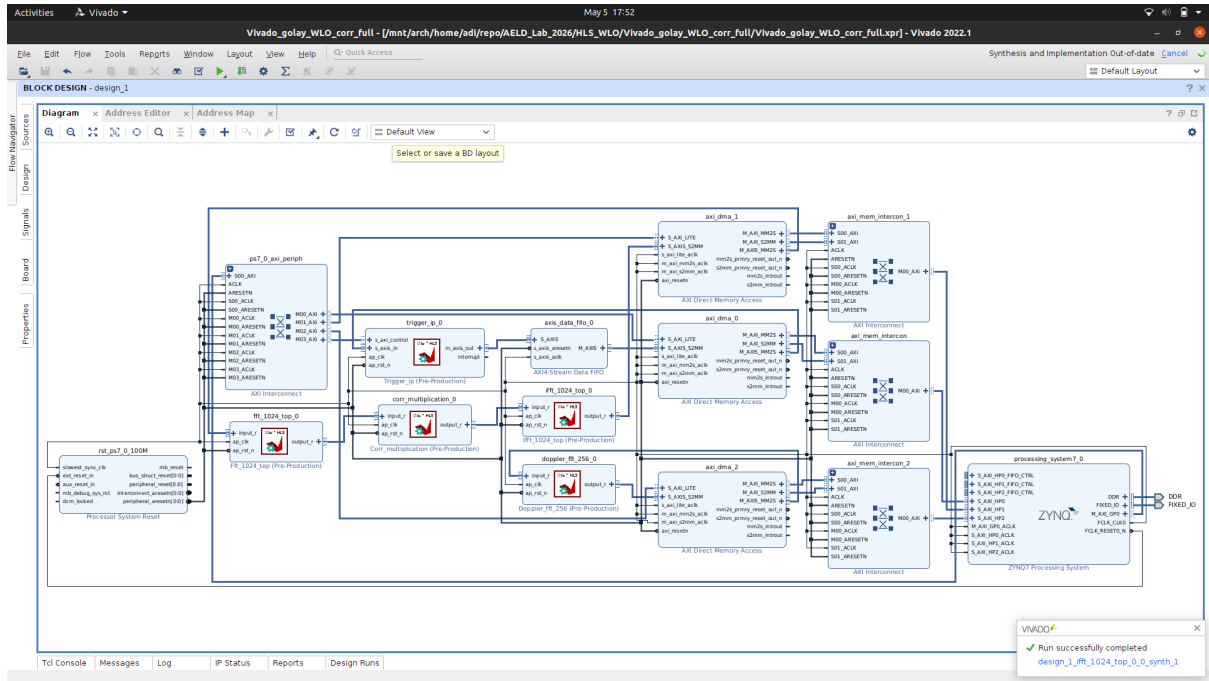


Figure 43: Block Design for the Fixed Point Implementation

Finally, we verified the overall functionality of the optimized hardware. The terminal output confirms that despite the reduced bit precision in the calculations, the target range and velocity estimations remain accurate.

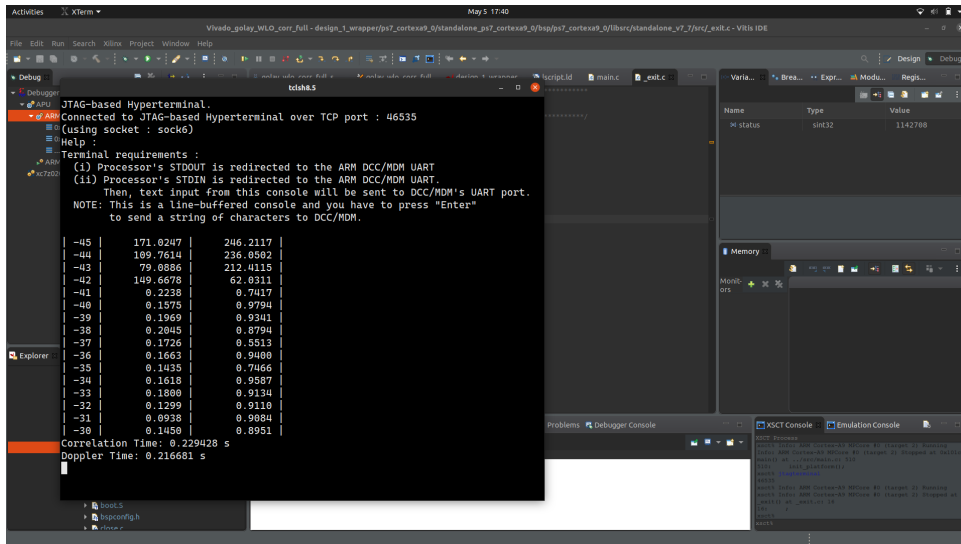


Figure 44: Validation Output for the Fixed Point Hardware

14.3.1 Iteration 2

In the second iteration of our word length optimization, we aggressively reduced the bit widths to save even more hardware resources. To achieve this without losing the required mathematical stability, we introduced a scaling shift factor for both the forward FFT and the multiplication blocks.

By applying a manual scale shift, we keep the data values small and avoid the need for large integer bit allocations. The updated module configurations for this second iteration are as follows:

- **Forward FFT:** We reduced the main data array to 26 total bits with only 2 integer bits. To make this small integer width work, a scale shift of 14 is applied to the input data. The twiddle factors are adjusted to use 24 total bits with 2 integer bits.
- **Inverse FFT:** The total bit width for the main array is further reduced to 24 bits with 2 integer bits. The twiddle factors match the forward FFT and use 24 total bits with 2 integer bits.
- **Multiplication Block:** The standard correlation data is reduced to 27 total bits and 9 integer bits. This stage utilizes a new scale shift of 11 to normalize the values. The conjugate reference data remains at 27 total bits with 11 integer bits.

This second iteration represents a highly tuned fixed point architecture. The heavy reduction in total bits and the smart use of input scaling allow the design to use the absolute minimum number of logical resources. By squeezing the data types down to these precise limits, the hardware footprint is fully minimized while keeping the target detection functional.

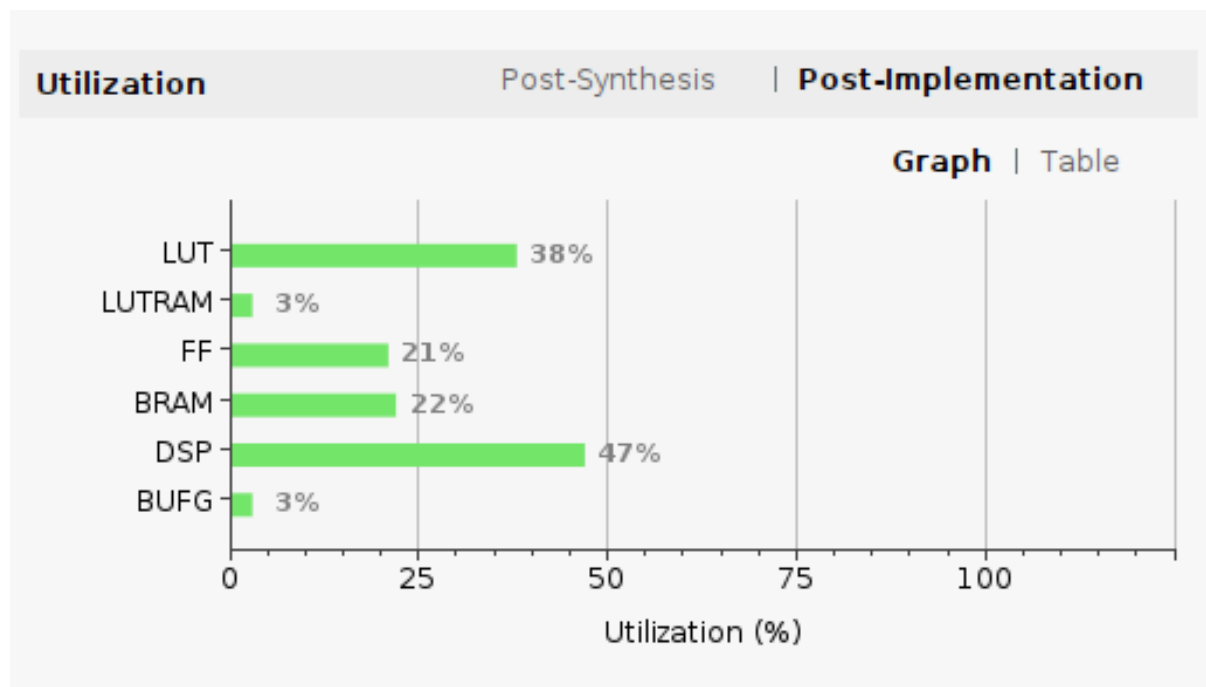


Figure 45: Utilization for Optimized Word Length IPs

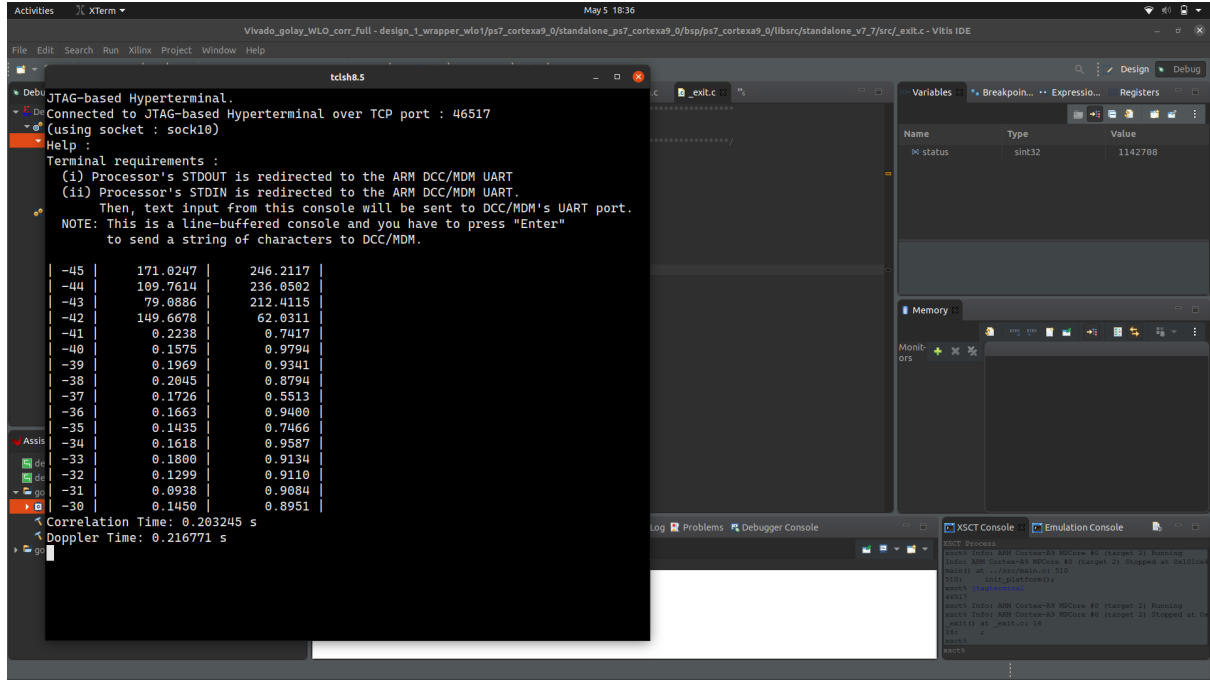


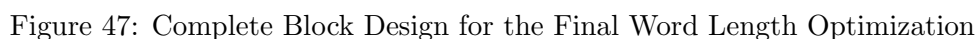
Figure 46: Final timing for the Word Length Optimized IP for GOLay is 419ms

14.3.2 Iteration 3

This was the final iteration of our word length optimization process. This specific configuration achieved the best overall speedup with respect to the total calculation time. In this phase, we took the highly tuned modules from the second iteration and applied the same strict fixed point strategy to the separate Doppler processing block.

For the 256 point Doppler FFT, we configured the custom data types to use the absolute minimum sizes required to maximize speed and resource efficiency. The main Doppler data array uses 19 total bits with only 2 integer bits. To make this precision work without causing mathematical overflow, we implemented a manual scale shift of 13. The precomputed twiddle factors for the Doppler block were set to use 24 total bits with 2 integer bits.

By combining these newly optimized Doppler settings with the aggressive configurations from the second iteration, we created the fastest and most efficient version of our hardware design. The figures below show the complete Vivado block design for this final word length optimized system and the terminal output confirming the successful processing results.



For Zadoff Chu

Execution Time: 302 ms

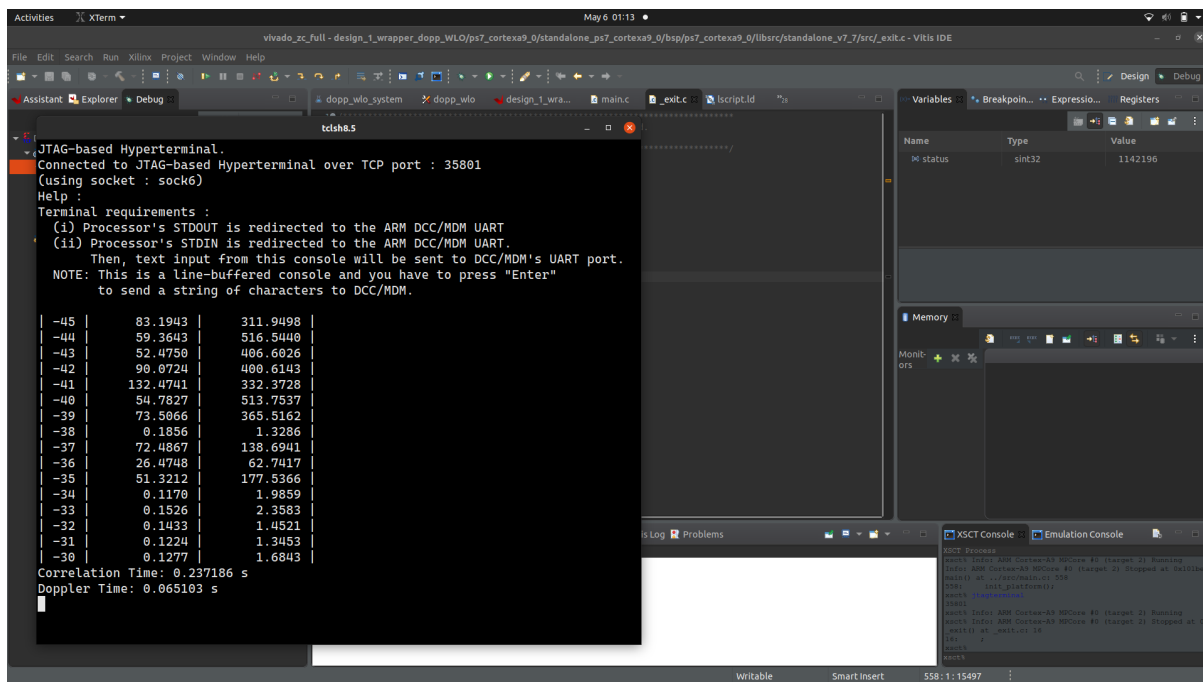


Figure 49: Zadoff Chu Execution Time with Doppler Word Length Optimization

We also tried to use the ACP port for the doppler IP due to the smaller size of samples (256 samples); however there was no significant difference in the execution time.

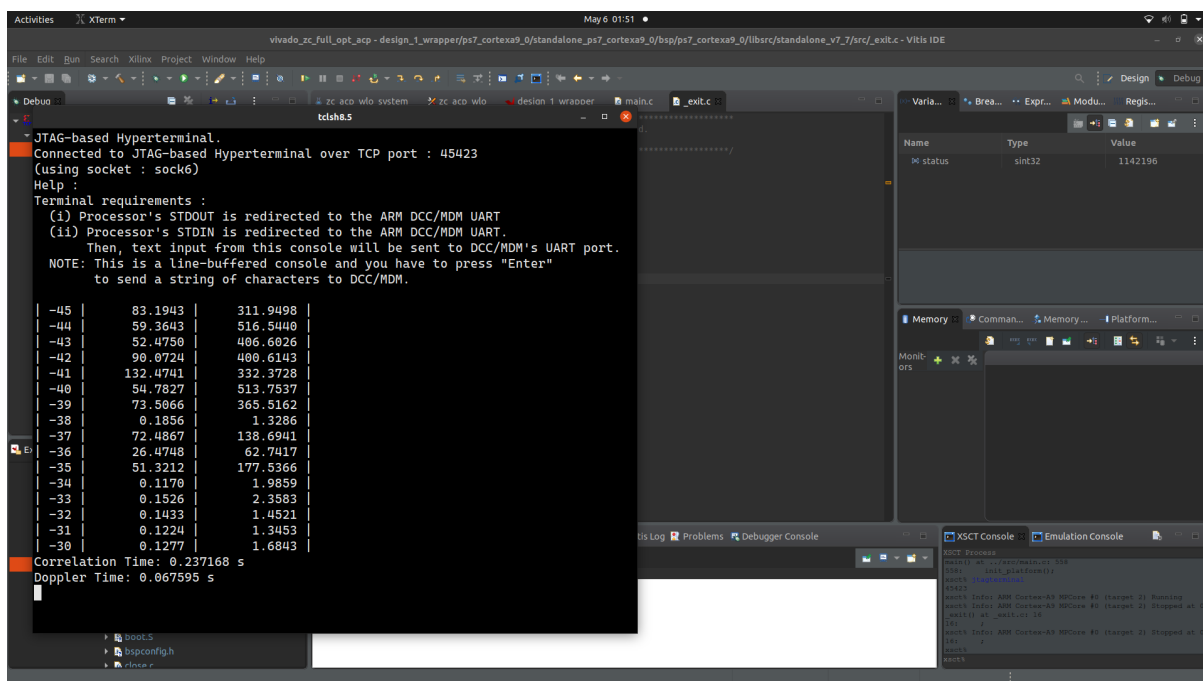


Figure 50: Zadoff Chu Execution Time with ACP port connection

14.4 Execution Times

14.4.1 Golay

Stage	Golay Execution Time
Base	771 ms
Pragma Optimized	1.25 s
Base on ZC706	1.47 s
With Pragmas on ZC706	2.37 s
Relaxed Correlation Word Length Optimization	445 ms
Hardened Correlation Word Length Optimization	419 ms
Correlation and Doppler Word Length Optimization	333 ms
PS (most optimal)	352 ms

14.4.2 Zadoff Chu

Stage	Zadoff-Chu Execution Time
Base	345 ms
Memory Mapped	366 ms
Doppler Word Length Optimization	302 ms
ACP Port	303 ms
PS (most optimal)	204 ms

14.5 Plots

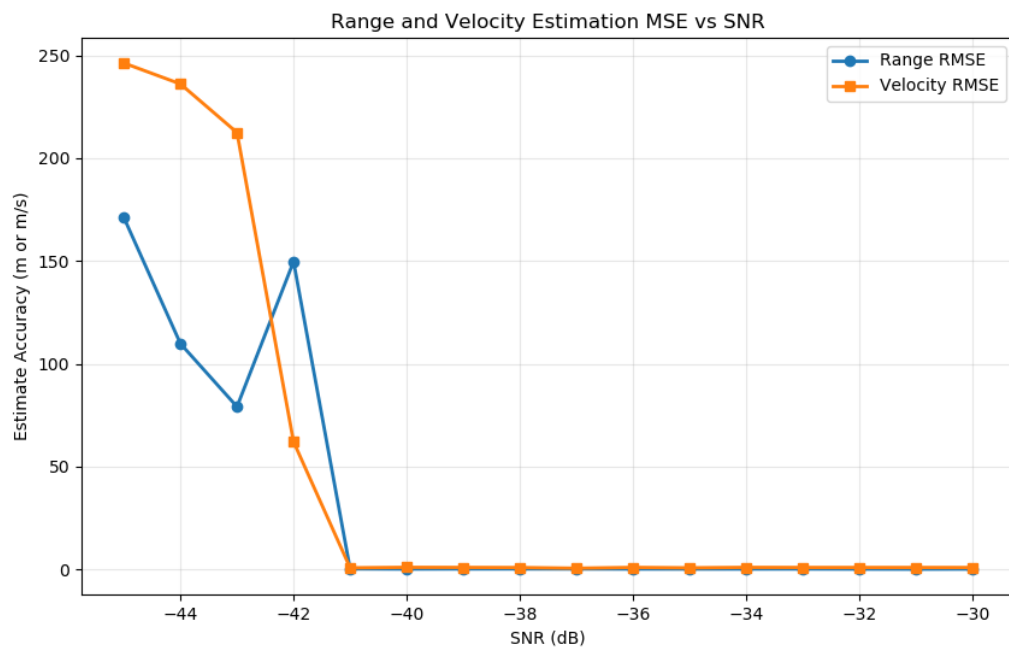


Figure 51: Golay RMSE Plots

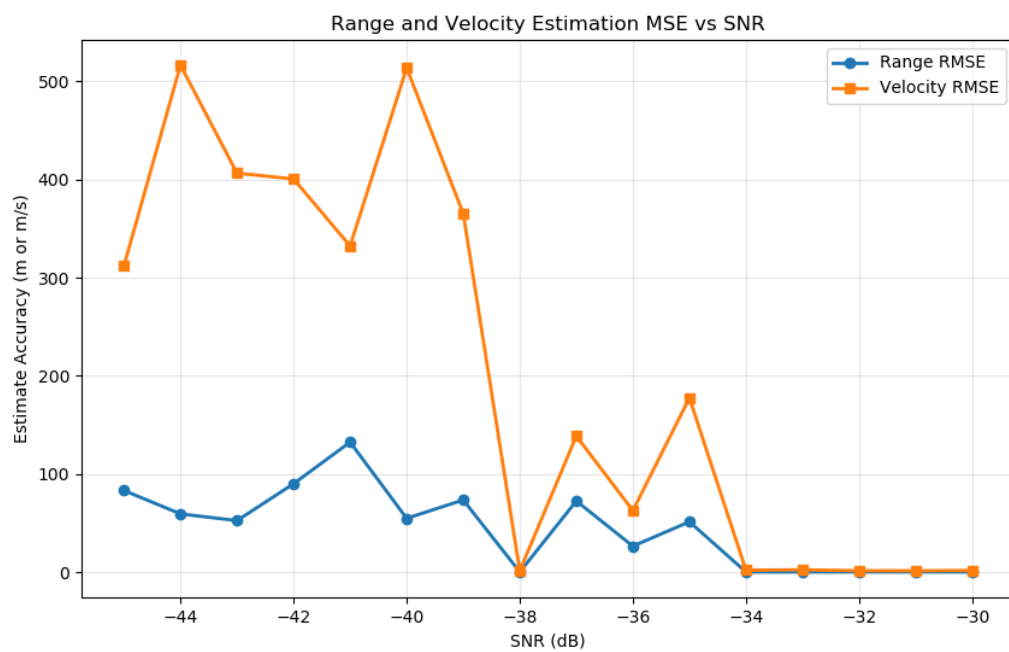


Figure 52: Zadoff Chu RMSE Plots

Distribution of work

Aditya Gautam :

Vitis IDE Code, FFT-1024, IFFT-1024, Doppler

Aditya Kak :

Multiplication, Doppler, Golay Memory Mapped, Report

Kabeer Kundra :

FFT-512, IFFT-512, Golay Memory Mapped, Report

Nadish Gulati :

Trigger IP, Word Length Optimization, ILA, Pragmas